LAPORAN UJIAN TENGAH SEMESTER

Mobile Programming

Rizal Docs

Aplikasi Repositori & Koleksi Dokumen

Nama Lengkap	**RIZAL SURYAWAN**
NIM	**241011750067**
Kelas	**04SIFE002**
Program Studi	Sistem Informasi S-1
Kampus	Universitas Pamulang
Tema Aplikasi	Repositori / Koleksi Dokumen (NIM berakhiran 7)



PROGRAM STUDI SISTEM INFORMASI S-1
UNIVERSITAS PAMULANG
2026

Daftar Isi

1. BAB 1 — Pendahuluan
2. BAB 2 — Deskripsi Aplikasi
3. BAB 3 — Prosedur Pengerjaan
   Tahap 1 – 17: dokumentasi antarmuka dan source code
4. BAB 4 — Hasil Akhir
5. BAB 5 — Kesimpulan
6. Lampiran A — Source Code Aplikasi (folder lib)
7. Lampiran B — Source Code Pengujian
8. Lampiran C — Repositori dan Riwayat Commit

> **Cara membaca laporan:** Bab 3 memuat screenshot setiap tahapan antarmuka aplikasi beserta keterangan dan potongan source code yang relevan. Source code lengkap dilampirkan pada Lampiran A dan B, sedangkan project akhir dapat dijalankan dari repositori GitHub pada Lampiran C.

BAB 1 · PENDAHULUAN

1.1 Latar Belakang

Ujian Tengah Semester (UTS) mata kuliah Mobile Programming menuntut mahasiswa membangun aplikasi mobile secara utuh: mulai dari perancangan antarmuka, navigasi antar halaman, pengelolaan data, sampai pengujian. Aplikasi yang dibuat pada laporan ini bernama **Rizal Docs**, yaitu aplikasi repositori dan koleksi dokumen yang berjalan pada platform Android dengan framework Flutter.

Aplikasi ini menyimpan data pengguna dan data dokumen pada database lokal SQLite sehingga seluruh fitur dapat digunakan tanpa koneksi internet. Gambar profil, logo, dan sampul dokumen juga berasal dari aset lokal yang disertakan di dalam project, bukan diunduh dari internet.

1.2 Tema Aplikasi

Tema aplikasi ditentukan dari angka terakhir NIM. NIM **241011750067** berakhiran angka **7**, sehingga tema yang dipilih adalah **Aplikasi Repositori / Koleksi Dokumen**. Tema tersebut diwujudkan dalam bentuk katalog dokumen: setiap dokumen memiliki judul, penulis, kategori, tahun, deskripsi, sampul, dan status favorit.

1.3 Tujuan

- Membangun aplikasi Android berbasis Flutter sesuai tema repositori dokumen.
- Menerapkan autentikasi (registrasi dan login) dengan validasi form dan SQLite.
- Membangun dashboard dengan header, body yang dapat di-scroll, dan bottom navigation.
- Menerapkan navigasi detail, aksi favorit dengan dialog konfirmasi, serta CRUD dokumen.
- Mendokumentasikan setiap tahapan pembuatan antarmuka beserta source code aplikasi.

1.4 Alat dan Bahan

Komponen	Keterangan
Framework	Flutter 3.47.7 (stable) dengan bahasa Dart 3.13.5
Penyimpanan data	SQLite melalui package `sqflite` dan `path`
Format tanggal	Package `intl` dengan locale `id_ID`
Perangkat uji	Android Emulator (resolusi 1080 x 2400)
Aset gambar	Dibuat lokal melalui skrip `tool/generate_assets.ps1`
Pengujian	`flutter analyze`, `flutter test`, dan integration test
Pembuatan laporan	HTML + Microsoft Edge headless (print to PDF)

1.5 Identitas Mahasiswa

Nama Lengkap	RIZAL SURYAWAN
NIM	241011750067
Kelas	04SIFE002
Program Studi	Sistem Informasi S-1
Kampus	Universitas Pamulang

1.6 Sistematika Penulisan

Laporan ini disusun dengan urutan sebagai berikut agar pembaca dapat mengikuti proses pengerjaan dari awal sampai akhir.

- **Bab 1 Pendahuluan:** latar belakang, tema aplikasi, tujuan, alat dan bahan, identitas mahasiswa, serta sistematika penulisan.
- **Bab 2 Deskripsi Aplikasi:** gambaran umum, fitur, pemenuhan ketentuan soal, struktur project, struktur folder aset, skema database, dan alur aplikasi.
- **Bab 3 Prosedur Pengerjaan:** dokumentasi tujuh belas tahap pembuatan antarmuka beserta screenshot, potongan source code, dan hasil pengujian.
- **Bab 4 Hasil Akhir:** ringkasan hasil per soal, galeri tampilan akhir, dan informasi build aplikasi.
- **Bab 5 Kesimpulan:** rangkuman capaian dan saran pengembangan.
- **Lampiran A–C:** source code aplikasi, source code pengujian, serta informasi repositori dan riwayat commit.

BAB 2 · DESKRIPSI APLIKASI

2.1 Gambaran Umum

Rizal Docs adalah aplikasi repositori dokumen. Pengguna harus registrasi dan login terlebih dahulu, kemudian dapat melihat koleksi dokumen, mencari dokumen, membuka detail, menandai favorit, serta menambah, mengubah, dan menghapus dokumen melalui menu Pengaturan. Aplikasi terdiri dari halaman splash, autentikasi, dashboard dengan empat tab, halaman detail, form dokumen, dan halaman profil.

2.2 Fitur Utama

Fitur	Penjelasan	Lokasi Kode
Registrasi	Membuat akun baru dengan nama lengkap, username, password, dan konfirmasi password.	`features/auth/register_page.dart`
Login	Memvalidasi username dan password ke database, menampilkan SnackBar bila berhasil dan AlertDialog bila gagal.	`features/auth/login_page.dart`
Dashboard	Header AppBar, kartu sambutan, pencarian, filter kategori, dan daftar dokumen yang dapat di-scroll.	`features/dashboard/`
Detail dokumen	Menampilkan sampul, judul, kategori, tahun, penulis, deskripsi, dan informasi tambahan.	`features/documents/document_detail_page.dart`
Favorit	Menandai atau menghapus dokumen dari favorit melalui dialog konfirmasi dan SnackBar.	`document_detail_page.dart`
CRUD dokumen	Tambah, ubah, dan hapus dokumen dari menu Pengaturan beserta dialog konfirmasi hapus.	`features/settings/settings_page.dart`
Profil	Menampilkan identitas mahasiswa, data akun, dan tombol keluar.	`features/profile/profile_page.dart`

2.3 Pemenuhan Ketentuan Soal

Soal	Ketentuan	Implementasi
1 (20%)	Nama aplikasi sesuai mahasiswa, foto profil bertema pada login, seluruh gambar dari aset lokal, struktur folder aset terdokumentasi.	Nama **Rizal Docs**, gambar dari `assets/images/`, dokumentasi pada bagian 2.5.
2 (20%)	Login dan registrasi dengan `Form`, `GlobalKey<FormState>`, `TextFormField`, method `actionLogin()` dan `actionRegistrasi()`, validasi, SQLite, SnackBar, dan AlertDialog.	`login_page.dart`, `register_page.dart`, `validators.dart`, `auth_repository.dart`.
3 (25%)	Dashboard dengan AppBar, body scroll, BottomNavigationBar, StatefulWidget, daftar dari aset lokal, dan nama pengguna.	`dashboard_page.dart` dan `home_tab.dart`.
4 (25%)	Menu Pengaturan dengan CRUD, detail melalui `Navigator.push`, aksi favorit dengan dialog Ya/Batal dan SnackBar.	`settings_page.dart`, `document_form_page.dart`, `document_detail_page.dart`.

2.4 Struktur Project

```
rizal_apps/
+-- android/                    konfigurasi Android (label: Rizal Docs)
+-- assets/images/
|   +-- brand/logo_rizal_docs.png
|   +-- covers/cover_01.png ... cover_06.png
|   +-- profile/profile_rizal.png
+-- docs/
|   +-- screenshots/            screenshot dokumentasi (16 gambar)
|   +-- laporan/                laporan PDF ini
+-- integration_test/app_test.dart
+-- lib/
|   +-- main.dart               entry point + locale Indonesia
|   +-- app.dart                MaterialApp, tema, splash
|   +-- core/
|   |   +-- constants/          nama aplikasi, identitas, path aset
|   |   +-- database/           helper SQLite (users + documents)
|   |   +-- theme/              warna, tipografi, komponen
|   |   +-- utils/              validator dan format tanggal
|   |   +-- widgets/            field, tombol, dialog, SnackBar
|   +-- data/document_seed.dart 6 data dokumen awal
|   +-- models/                 User dan Document
|   +-- repositories/           AuthRepository, DocumentRepository
|   +-- features/
|       +-- splash/             splash screen
|       +-- auth/               login, registrasi, scaffold autentikasi
|       +-- dashboard/          dashboard + tab beranda
|       +-- documents/          detail, form CRUD, kartu dokumen
|       +-- favorites/          tab favorit
|       +-- profile/            profil mahasiswa
|       +-- settings/           pengelolaan dokumen
+-- test/                       unit test dan widget test
+-- tool/generate_assets.ps1    generator aset gambar lokal
+-- pubspec.yaml
```

2.5 Struktur Folder Aset

Seluruh gambar berada di dalam folder `assets/images/` dan didaftarkan pada `pubspec.yaml`. Tidak ada pemanggilan `Image.network`, `NetworkImage`, atau URL internet pada kode aplikasi.

File	Penggunaan
`assets/images/profile/profile_rizal.png`	Foto profil bertema pada halaman login, dashboard, dan profil.
`assets/images/brand/logo_rizal_docs.png`	Logo aplikasi pada splash, AppBar dashboard, dan sampul laporan.
`assets/images/covers/cover_01.png`	Sampul dokumen Bisnis Digital dan Kewirausahaan.
`assets/images/covers/cover_02.png`	Sampul dokumen Kecerdasan Buatan Terapan.
`assets/images/covers/cover_03.png`	Sampul dokumen Manajemen Proyek Perangkat Lunak.
`assets/images/covers/cover_04.png`	Sampul dokumen Basis Data Relasional.
`assets/images/covers/cover_05.png`	Sampul dokumen Metodologi Penelitian Sistem Informasi.
`assets/images/covers/cover_06.png`	Sampul dokumen Dasar-Dasar Pemrograman Mobile.

Registrasi aset pada pubspec.yaml: `assets/images/profile/`, `assets/images/brand/`, dan `assets/images/covers/`.

2.6 Skema Database

Database bernama `rizal_docs.db` dengan versi 2. Versi 1 membuat tabel `users`, sedangkan versi 2 menambahkan tabel `documents` beserta enam data awal. Migrasi ditangani oleh callback `onUpgrade` agar data pengguna tidak hilang ketika aplikasi diperbarui.

Tabel users	Tipe	Keterangan
id	INTEGER	Primary key autoincrement
nama	TEXT	Nama lengkap pengguna
username	TEXT	Unik, dipakai untuk login
password	TEXT	Password pengguna
created_at	TEXT	Waktu pendaftaran (ISO 8601)

Tabel documents	Tipe	Keterangan
id	INTEGER	Primary key autoincrement
title	TEXT	Judul dokumen
category	TEXT	Kategori dokumen
author	TEXT	Penulis dokumen
year	INTEGER	Tahun terbit
description	TEXT	Deskripsi dokumen
image	TEXT	Path sampul di folder aset lokal
is_favorite	INTEGER	Status favorit: 0 atau 1
created_at	TEXT	Waktu dibuat
updated_at	TEXT	Waktu diubah

Pembuatan tabel users dan documents — `lib/core/database/database_helper.dart` (baris 54–81)

```
Future<void> _createUsersTable(Database db) async {
  await db.execute('''
    CREATE TABLE ${AppConstants.usersTable} (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nama TEXT NOT NULL,
      username TEXT NOT NULL UNIQUE,
      password TEXT NOT NULL,
      created_at TEXT NOT NULL
    )
  ''');
}

Future<void> _createDocumentsTable(Database db) async {
  await db.execute('''
    CREATE TABLE ${AppConstants.documentsTable} (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      category TEXT NOT NULL,
      author TEXT NOT NULL,
      year INTEGER NOT NULL,
      description TEXT NOT NULL,
      image TEXT NOT NULL,
      is_favorite INTEGER NOT NULL DEFAULT 0,
      created_at TEXT NOT NULL,
      updated_at TEXT NOT NULL
    )
  ''');
}
```

2.7 Alur Aplikasi



Selain jalur utama di atas, menu Pengaturan menyediakan jalur CRUD: daftar dokumen, form tambah, form ubah, dan dialog hapus. Semua perubahan langsung disimpan ke SQLite dan daftar dimuat ulang setelah halaman kembali dibuka.

2.8 Peta Navigasi

Dari	Aksi Pengguna	Halaman Tujuan	Metode Navigasi
Login	Login berhasil	Dashboard	`pushReplacement`
Dashboard	Ketuk kartu dokumen	Detail Dokumen	`Navigator.push`
Detail	Tombol tambah/hapus favorit	Dialog konfirmasi	`showDialog`
Detail	Kembali ke daftar	Dashboard (muat ulang)	`Navigator.pop`
Pengaturan	Tambah atau ubah dokumen	Form Dokumen	`push<bool>`
Pengaturan	Hapus dokumen	Dialog konfirmasi	`showDialog`
Profil	Keluar dari akun	Login	`pushAndRemoveUntil`

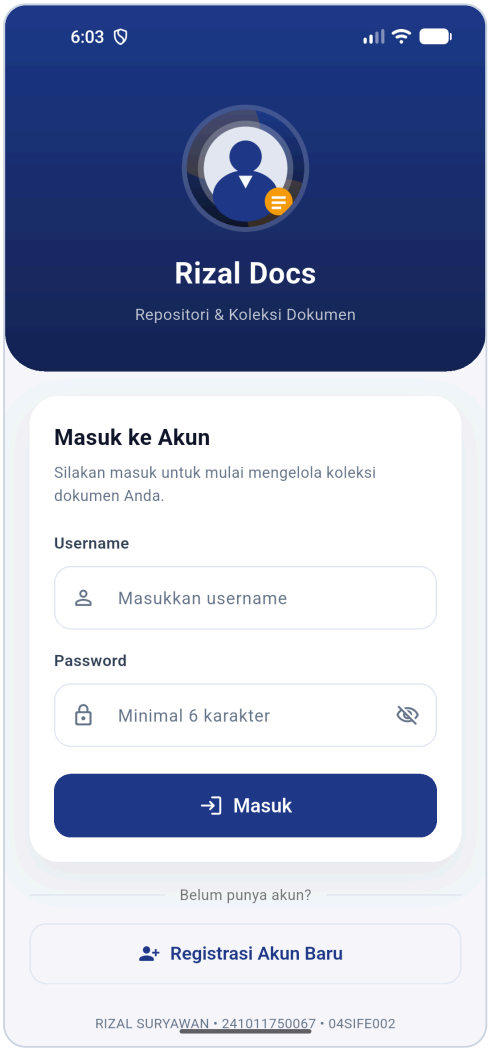
BAB 3 · PROSEDUR PENGERJAAN

Bab ini mendokumentasikan tahapan pembuatan antarmuka aplikasi. Setiap tahap disertai screenshot dari emulator Android (resolusi 1080 x 2400) dan keterangan mengenai komponen Flutter yang digunakan. Potongan source code ditampilkan pada tahap yang relevan, sedangkan source code lengkap berada pada Lampiran A dan B.

> **Ringkasan tahapan:** (1) inisialisasi project dan tema, (2) pembuatan aset lokal, (3) pembuatan database SQLite, (4) halaman splash dan login, (5) registrasi, (6) validasi form, (7) dashboard, (8) detail dokumen, (9) favorit, (10) CRUD dokumen, (11) profil, dan (12) pengujian.

Tahap 1 — Halaman Login

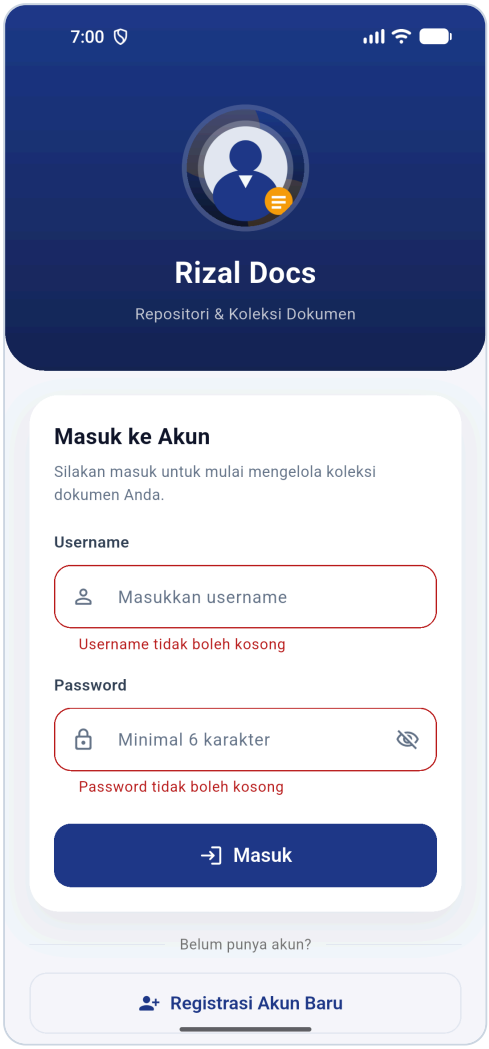
Halaman login dibuat dengan `AuthScaffold` yang menampilkan foto profil bertema repositori dokumen, nama aplikasi, dan identitas mahasiswa di bagian bawah. Form login menggunakan dua buah `TextFormField` (username dan password) di dalam `Form` dengan `GlobalKey<FormState>`. Tombol *Masuk* memanggil method `actionLogin()`.



Gambar 3.1 Halaman login
Foto profil, judul aplikasi, form login, dan identitas mahasiswa.

Tahap 2 — Validasi Form Login

Validasi dijalankan sebelum data dikirim ke database. Jika username atau password kosong, pesan kesalahan muncul tepat di bawah field masing-masing. Password juga divalidasi minimal enam karakter. Validator dipusatkan pada class `AppValidators` agar konsisten dipakai di halaman login dan registrasi.



Gambar 3.2 Validasi form login
Pesan Username tidak boleh kosong dan Password tidak boleh kosong tampil di bawah field.

Validator username dan password — `lib/core/utils/validators.dart` (baris 1–66)

```
/// Kumpulan validator form yang dipakai pada halaman login dan registrasi.
///
/// Dipisahkan dari widget agar mudah diuji dan dipakai ulang.
class AppValidators {
  const AppValidators._();

  static String? nama(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Nama lengkap tidak boleh kosong';
    }
    if (value.trim().length < 3) {
      return 'Nama lengkap minimal 3 karakter';
    }
    return null;
  }

  static String? username(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Username tidak boleh kosong';
```

```
    }
    return null;
  }

  static String? password(String? value) {
    if (value == null || value.isEmpty) {
      return 'Password tidak boleh kosong';
    }
    if (value.length < 6) {
      return 'Password minimal 6 karakter';
    }
    return null;
  }

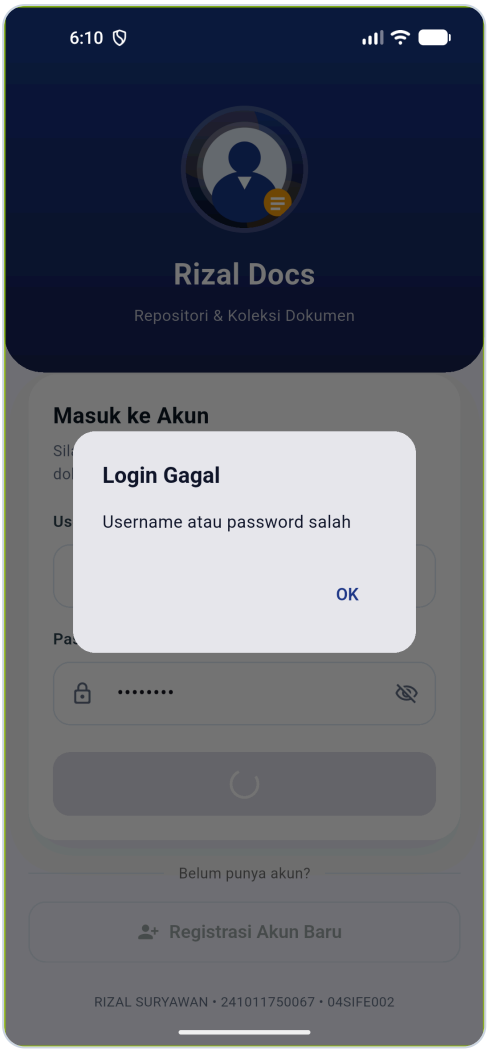
  static String? konfirmasiPassword(String? value, String password) {
    if (value == null || value.isEmpty) {
      return 'Konfirmasi password tidak boleh kosong';
    }
    if (value != password) {
      return 'Konfirmasi password tidak sesuai';
    }
    return null;
  }

  /// Validator umum untuk field teks yang wajib diisi.
  static String? requiredField(String? value, String label) {
    if (value == null || value.trim().isEmpty) {
      return '$label tidak boleh kosong';
    }
    return null;
  }

  /// Validator tahun terbit dokumen (1900 - 2100).
  static String? tahun(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Tahun tidak boleh kosong';
    }
    final int? year = int.tryParse(value.trim());
    if (year == null) {
      return 'Tahun harus berupa angka';
    }
    if (year < 1900 || year > 2100) {
      return 'Tahun harus di antara 1900 dan 2100';
    }
    return null;
  }
}
```

Tahap 3 — Login Gagal

Apabila kombinasi username dan password tidak ditemukan pada tabel `users`, aplikasi menampilkan `AlertDialog` berjudul *Login Gagal* dengan pesan *Username atau password salah* dan satu tombol *OK*. Dialog dibuat melalui helper `showAppDialog()` agar tampilannya seragam.



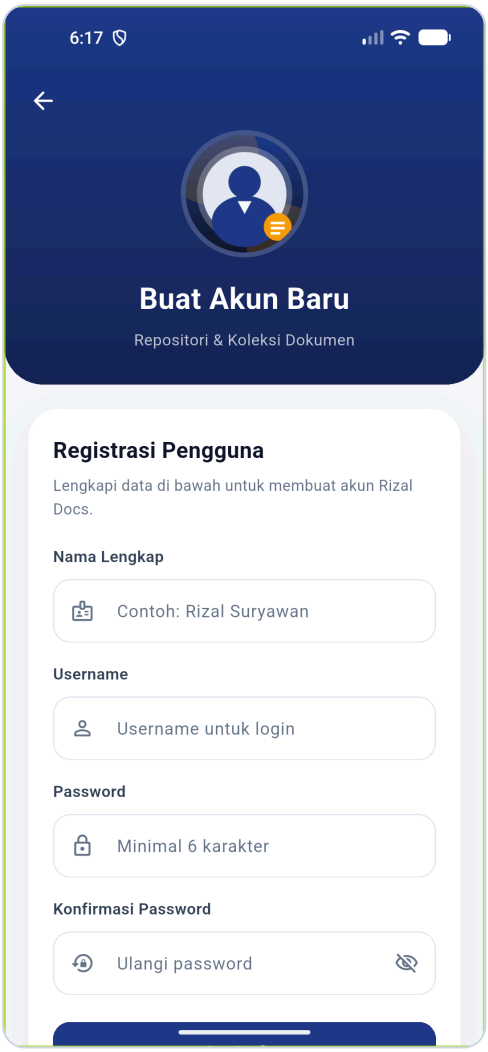
Gambar 3.3 Dialog login gagal
AlertDialog dengan pesan Username atau password salah dan tombol OK.

Penanganan login gagal — `lib/features/auth/login_page.dart` (baris 53–60)

```
if (user == null) {
  await showAppDialog(
    context,
    title: 'Login Gagal',
    message: 'Username atau password salah',
  );
  return;
}
```

Tahap 4 — Form Registrasi

Halaman registrasi menggunakan pola yang sama: `Form`, `GlobalKey<FormState>`, dan empat `TextFormField` untuk nama lengkap, username, password, dan konfirmasi password. Method `actionRegistrasi()` memeriksa keunikan username, menyimpan data ke SQLite, lalu kembali ke halaman login dengan membawa username yang baru didaftarkan.



Gambar 3.4 Form registrasi akun
Empat field dengan validasi dan tombol Daftar.

Method actionRegistrasi() — `lib/features/auth/register_page.dart` (baris 38–82)

```
/// Memvalidasi form lalu menyimpan pengguna baru ke database SQLite.
Future<void> actionRegistrasi() async {
  FocusScope.of(context).unfocus();

  if (!(_formKey.currentState?.validate() ?? false)) return;

  setState(() => _isLoading = true);

  try {
    final String username = _usernameController.text.trim().toLowerCase();
    final bool isTaken = await AuthRepository.instance.isUsernameTaken(
      username,
    );

    if (!mounted) return;

    if (isTaken) {
      await showAppDialog(
        context,
        title: 'Registrasi Gagal',
        message: 'Username sudah digunakan, silakan pilih username lain.',
      );
      return;
    }

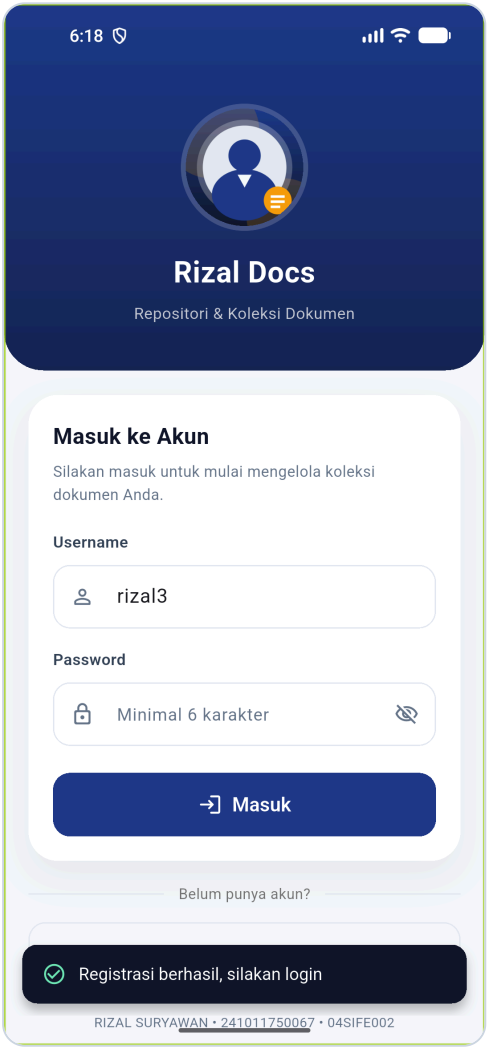
    final User user = await AuthRepository.instance.register(
      nama: _namaController.text,
```

```
        username: username,
        password: _passwordController.text,
      );

      if (!mounted) return;
      showAppSnackBar(context, 'Registrasi berhasil, silakan login');
      Navigator.of(context).pop(user.username);
    } catch (error) {
      if (!mounted) return;
      showAppDialog(
        context,
        title: 'Terjadi Kesalahan',
        message: 'Tidak dapat menyimpan data: $error',
      );
    } finally {
      if (mounted) setState(() => _isLoading = false);
    }
  }
```

Tahap 5 — Registrasi Berhasil

Setelah data tersimpan, aplikasi menampilkan SnackBar *Registrasi berhasil, silakan login* dan kembali ke halaman login. Username hasil registrasi otomatis diisikan pada field username agar pengguna tidak perlu mengetik ulang.

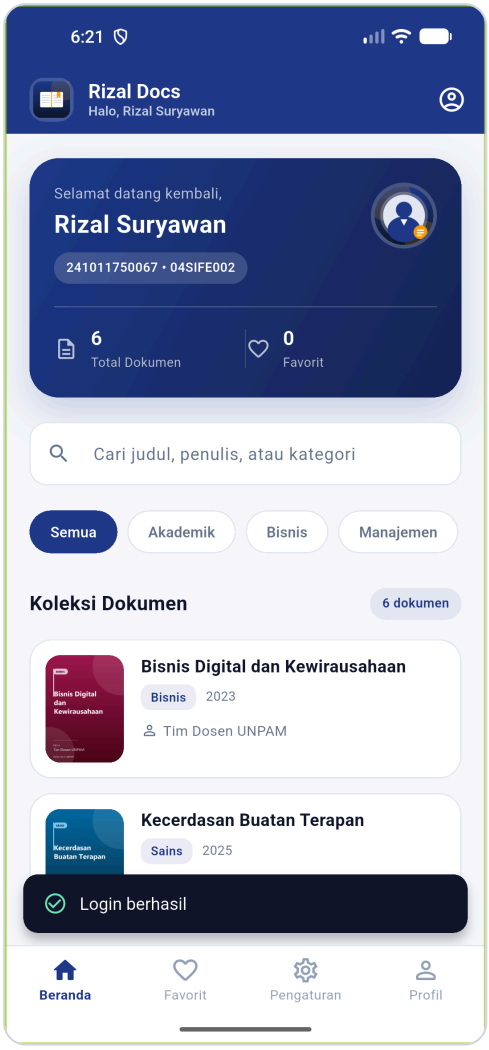


Gambar 3.5 Registrasi berhasil

SnackBar konfirmasi dan username terisi otomatis pada halaman login.

Tahap 6 — Login Berhasil

Jika autentikasi berhasil, method `actionLogin()` menampilkan SnackBar *Login berhasil* lalu berpindah ke dashboard menggunakan `Navigator.pushReplacement`. Data pengguna yang login dikirim ke `DashboardPage` sehingga namanya dapat ditampilkan pada header.



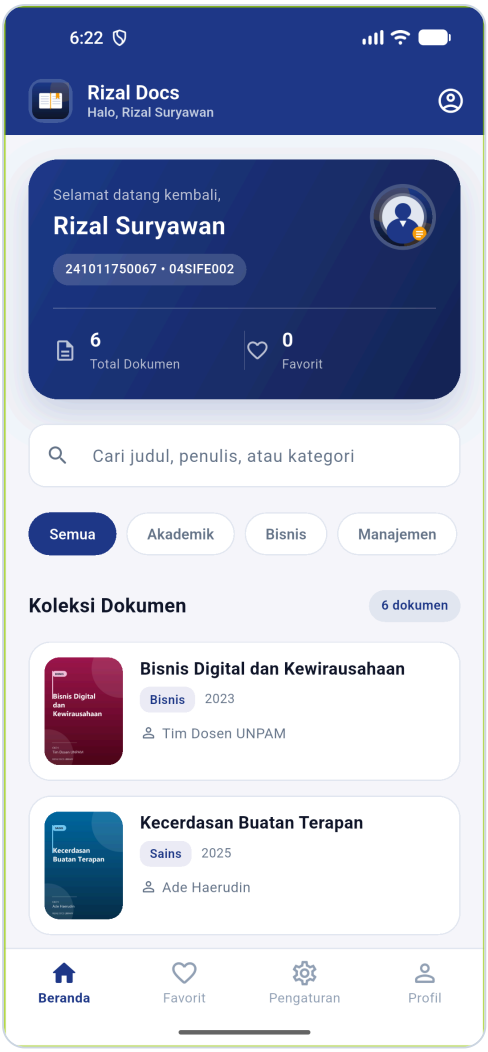
Gambar 3.6 Login berhasil
SnackBar Login berhasil dan dashboard menampilkan nama pengguna.

SnackBar dan navigasi ke dashboard — `lib/features/auth/login_page.dart` (baris 62–67)

```
showAppSnackBar(context, 'Login berhasil');
Navigator.of(context).pushReplacement(
  MaterialPageRoute<void>(
    builder: (BuildContext context) => DashboardPage(user: user),
  ),
);
```

Tahap 7 — Dashboard

Dashboard merupakan `StatefulWidget` yang menyimpan indeks tab aktif. Halaman terdiri dari tiga bagian sesuai ketentuan soal: *AppBar* sebagai header, *body* yang dapat di-scroll, dan `BottomNavigationBar` sebagai footer dengan empat tab (Beranda, Favorit, Pengaturan, Profil). Daftar dokumen pada tab Beranda dibangun dengan `ListView.builder` dan menampilkan nama pengguna yang sedang login pada sapaan *Halo, ….*



Gambar 3.7 Dashboard aplikasi

AppBar, kartu sambutan berisi nama dan NIM, pencarian, filter kategori, serta daftar dokumen.

Struktur dashboard: AppBar, body, dan BottomNavigationBar — `lib/features/dashboard/dashboard_page.dart` (baris 46–146)

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      automaticallyImplyLeading: false,
      titleSpacing: 20,
      title: Row(
        children: [
          Container(
            width: 36,
            height: 36,
            padding: const EdgeInsets.all(3),
            decoration: BoxDecoration(
              color: Colors.white.withValues(alpha: 0.16),
              borderRadius: BorderRadius.circular(12),
            ),
            child: Image.asset(AppConstants.logoImage),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              mainAxisSize: MainAxisSize.min,
              children: [
                const Text(
                  AppConstants.appName,
                  style: TextStyle(
```

```
                    fontSize: 16,
                    fontWeight: FontWeight.w700,
                    color: Colors.white,
                  ),
                ),
                Text(
                  'Halo, ${widget.user.nama}',
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: TextStyle(
                    fontSize: 11,
                    color: Colors.white.withValues(alpha: 0.75),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
      actions: [
        IconButton(
          onPressed: () => _onDestinationSelected(3),
          icon: const Icon(Icons.account_circle_outlined),
          tooltip: 'Profil',
        ),
        const SizedBox(width: 4),
      ],
    ),
    body: SafeArea(child: _buildBody()),
    bottomNavigationBar: Container(
      decoration: const BoxDecoration(
        color: Colors.white,
        border: Border(top: BorderSide(color: AppTheme.border)),
      ),
      child: BottomNavigationBar(
        currentIndex: _selectedIndex,
        onTap: _onDestinationSelected,
        type: BottomNavigationBarType.fixed,
        backgroundColor: Colors.white,
        elevation: 0,
        selectedItemColor: AppTheme.primary,
        unselectedItemColor: const Color(0xFF94A3B8),
        selectedLabelStyle: const TextStyle(
          fontSize: 11,
          fontWeight: FontWeight.w700,
        ),
        unselectedLabelStyle: const TextStyle(fontSize: 11),
        items: const [
          BottomNavigationBarItem(
            icon: Icon(Icons.home_outlined),
            activeIcon: Icon(Icons.home_rounded),
            label: 'Beranda',
          ),
          BottomNavigationBarItem(
            icon: Icon(Icons.favorite_border_rounded),
            activeIcon: Icon(Icons.favorite_rounded),
            label: 'Favorit',
          ),
          BottomNavigationBarItem(
            icon: Icon(Icons.settings_outlined),
            activeIcon: Icon(Icons.settings_rounded),
            label: 'Pengaturan',
          ),
          BottomNavigationBarItem(
            icon: Icon(Icons.person_outline_rounded),
            activeIcon: Icon(Icons.person_rounded),
            label: 'Profil',
          ),
        ],
      ),
    ),
  );
```

```
  }
}
```

Daftar dokumen dengan ListView.builder — `lib/features/dashboard/widgets/home_tab.dart` (baris 119–142)

```
    return RefreshIndicator(
      onRefresh: _loadDocuments,
      color: AppTheme.primary,
      child: ListView.builder(
        padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
        physics: const AlwaysScrollableScrollPhysics(),
        itemCount: documents.length + 1,
        itemBuilder: (BuildContext context, int index) {
          if (index == 0) {
            return _buildHeader(documents);
          }

          final Document document = documents[index - 1];
          return Padding(
            padding: const EdgeInsets.only(bottom: 12),
            child: DocumentCard(
              document: document,
              onTap: () => _openDetail(document),
            ),
          );
        },
      ),
    );
  }
```

Tahap 8 — Detail Dokumen

Detail dokumen dibuka dari kartu daftar menggunakan `Navigator.of(context).push(MaterialPageRoute(...))`. Halaman detail memakai `CustomScrollView` dan `SliverAppBar` untuk menampilkan sampul, kemudian judul, kategori, tahun, penulis, deskripsi, dan informasi tambahan berupa tanggal penambahan serta status favorit.



Gambar 3.8 Halaman detail dokumen
Sampul, judul, kategori, tahun, penulis, dan deskripsi dokumen.

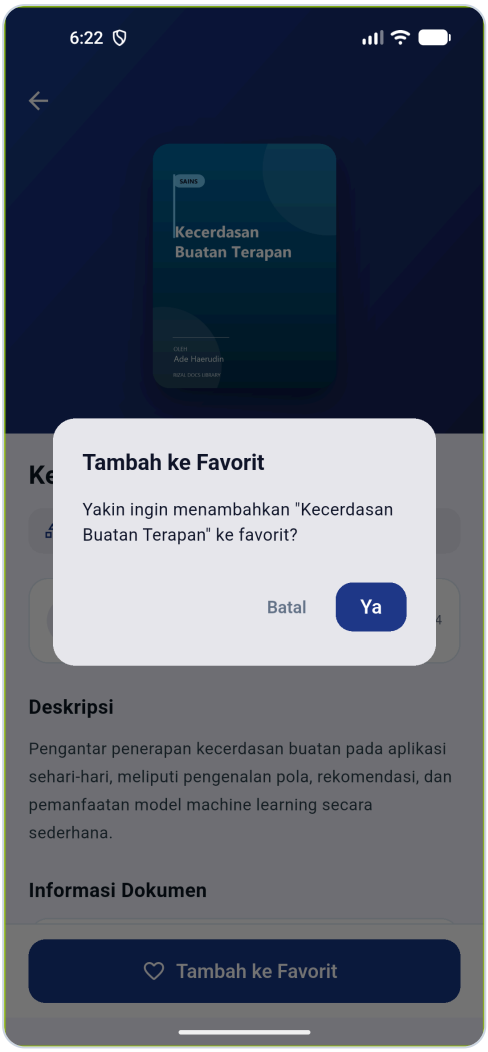
Navigasi ke halaman detail — `lib/features/dashboard/widgets/home_tab.dart` (baris 73–84)

```
Future<void> _openDetail(Document document) async {
  await Navigator.of(context).push(
    MaterialPageRoute<void>(
      builder: (BuildContext context) =>
          DocumentDetailPage(document: document),
    ),
  );

  if (!mounted) return;
  // Memuat ulang data agar status favorit pada daftar tetap sinkron.
  await _loadDocuments();
}
```

Tahap 9 — Konfirmasi Favorit

Tombol aksi pada halaman detail menampilkan teks *Tambah ke Favorit* atau *Hapus dari Favorit* sesuai status dokumen. Sebelum data diubah, aplikasi menampilkan `AlertDialog` konfirmasi dengan pesan *Yakin ingin menambahkan ... ke favorit?* serta tombol *Ya* dan *Batal*.



Gambar 3.9 Dialog konfirmasi favorit
Dialog Ya/Batal sebelum status favorit diubah.

Dialog konfirmasi sebelum mengubah favorit — `lib/features/documents/document_detail_page.dart` (baris 33–47)

```
Future<void> _toggleFavorite() async {
  final bool willAdd = !_document.isFavorite;

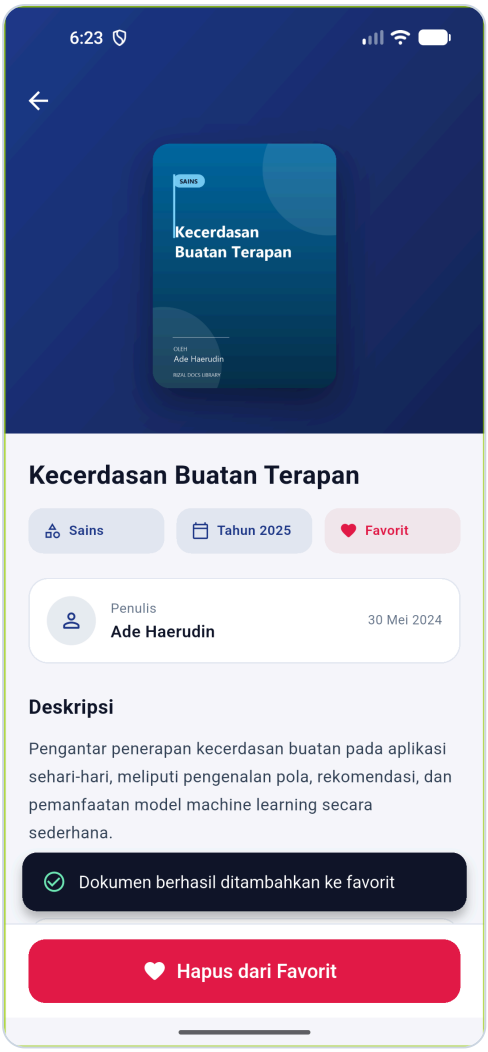
  final bool confirmed = await showConfirmDialog(
    context,
    title: willAdd ? 'Tambah ke Favorit' : 'Hapus dari Favorit',
    message: willAdd
        ? 'Yakin ingin menambahkan "${_document.title}" ke favorit?'
        : 'Yakin ingin menghapus "${_document.title}" dari favorit?',
  );

  if (!confirmed || !mounted) return;

  setState(() => _isUpdating = true);
```

Tahap 10 — Favorit Tersimpan

Jika pengguna memilih *Ya*, status `is_favorite` diperbarui pada database dan tampilan langsung berubah: tombol menjadi *Hapus dari Favorit* dan muncul SnackBar *Dokumen berhasil ditambahkan ke favorit*.



Gambar 3.10 Favorit berhasil disimpan
SnackBar konfirmasi dan tombol berubah menjadi Hapus dari Favorit.

Penyimpanan status favorit dan SnackBar — `lib/features/documents/document_detail_page.dart` (baris 54–67)

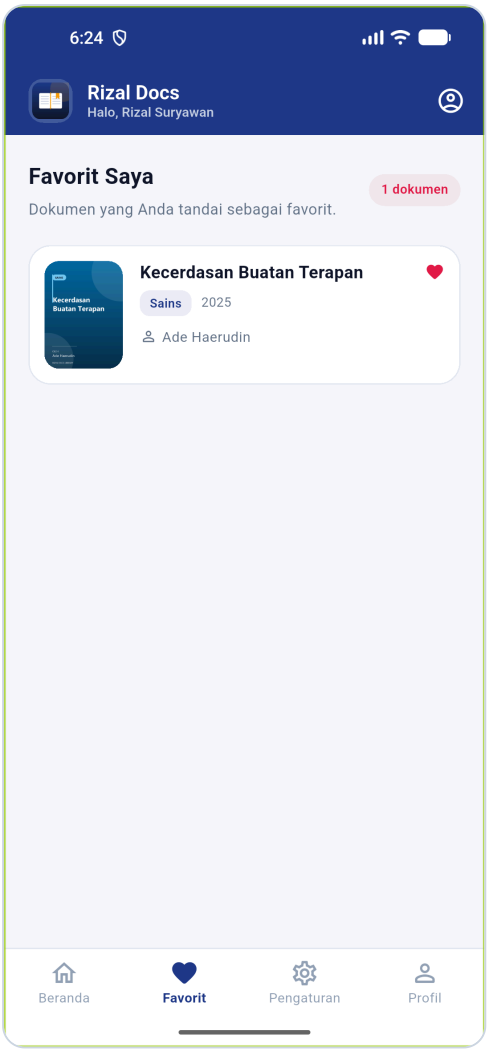
```
      await DocumentRepository.instance.setFavorite(id, willAdd);

      if (!mounted) return;
      setState(() {
        _document = _document.copyWith(isFavorite: willAdd);
        _isUpdating = false;
      });

      showAppSnackBar(
        context,
        willAdd
            ? 'Dokumen berhasil ditambahkan ke favorit'
            : 'Dokumen dihapus dari daftar favorit',
      );
```

Tahap 11 — Tab Favorit

Tab Favorit membaca dokumen dengan `is_favorite = 1` melalui `DocumentRepository.getFavorites()`. Daftar ditampilkan dengan `ListView.builder`, dilengkapi jumlah dokumen, pull-to-refresh, dan empty state apabila belum ada dokumen favorit.



Gambar 3.11 Tab favorit
Dokumen favorit beserta jumlah dan akses ke halaman detail.

Query dokumen favorit — `lib/repositories/document_repository.dart` (baris 21–30)

```
Future<List<Document>> getFavorites() async {
  final Database db = await DatabaseHelper.instance.database;
  final List<Map<String, Object?>> rows = await db.query(
    'documents',
    where: 'is_favorite = ?',
    whereArgs: [1],
    orderBy: 'created_at DESC, id DESC',
  );
  return rows.map(Document.fromMap).toList();
}
```

Tahap 12 — Menu Pengaturan (CRUD)

Menu Pengaturan memuat tombol *Tambah Dokumen* dan daftar seluruh dokumen. Setiap baris memiliki menu tindakan *Ubah* dan *Hapus*. Halaman ini juga menjelaskan alur create, read, update, dan delete kepada pengguna. Form tambah dan ubah dibuka melalui `Navigator.push` dan mengembalikan nilai `true` bila data berubah sehingga daftar otomatis dimuat ulang.



Gambar 3.12 Menu Pengaturan

Tombol tambah, daftar dokumen, dan menu ubah/hapus pada setiap baris.

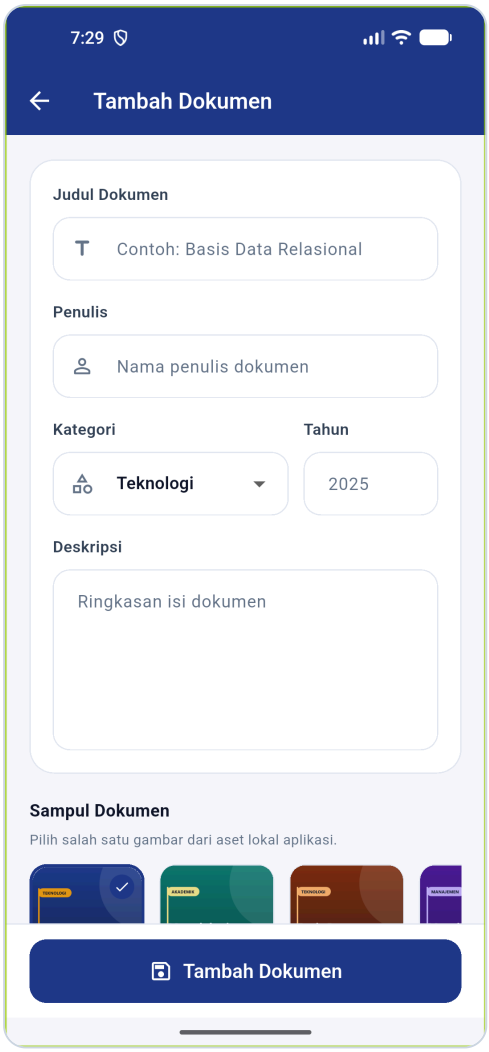
Membuka form tambah/ubah dokumen — `lib/features/settings/settings_page.dart` (baris 53–63)

```
Future<void> _openForm({Document? document}) async {
  final bool? changed = await Navigator.of(context).push<bool>(
    MaterialPageRoute<bool>(
      builder: (BuildContext context) => DocumentFormPage(document: document),
    ),
  );

  if (changed == true) {
    await _loadDocuments();
  }
}
```

Tahap 13 — Form Tambah Dokumen

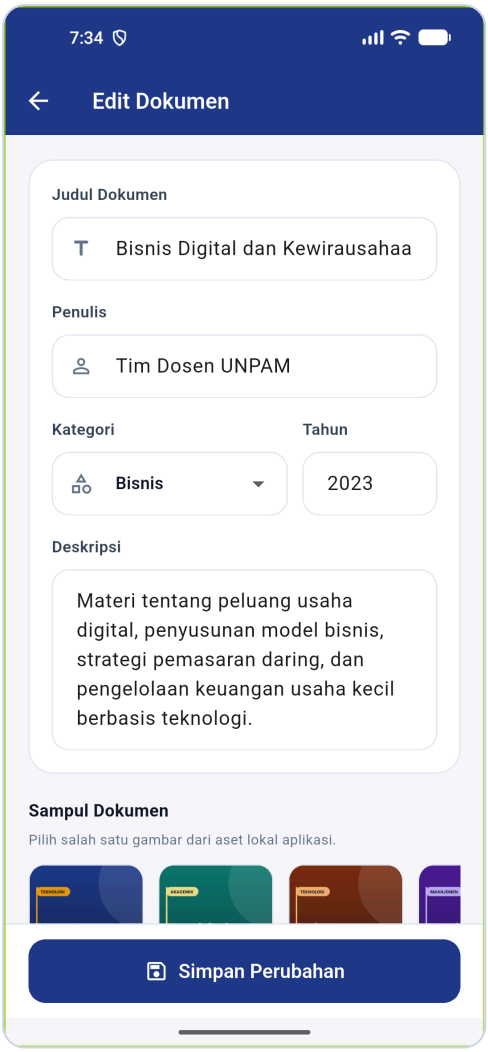
Form dokumen berisi field judul, penulis, kategori (dropdown), tahun, deskripsi, dan pemilih sampul dari enam aset lokal. Validasi memastikan judul, penulis, tahun, dan deskripsi tidak kosong, tahun berupa angka yang masuk akal, serta sampul telah dipilih.



Gambar 3.13 Form tambah dokumen
Field data dokumen dan pemilihan sampul dari aset lokal.

Tahap 14 — Form Ubah Dokumen

Form ubah memakai halaman yang sama dengan form tambah, tetapi menerima objek `Document` sehingga seluruh field terisi otomatis. Tombol berubah menjadi *Simpan Perubahan*, dan repository memperbarui kolom `updated_at`.



Gambar 3.14 Form ubah dokumen
Data dokumen terisi otomatis dan tombol Simpan Perubahan.

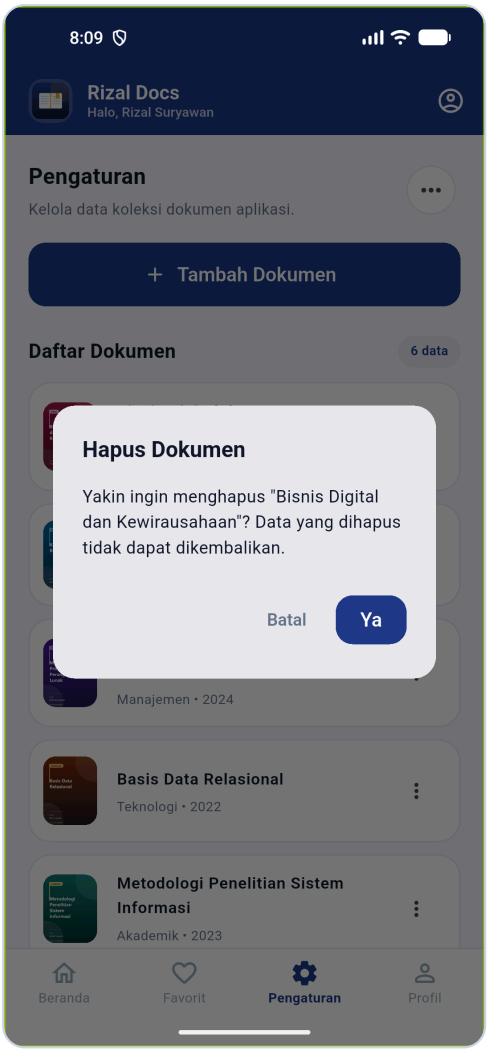
Update dokumen pada SQLite — `lib/repositories/document_repository.dart` (baris 59–72)

```
Future<void> update(Document document) async {
  final int? id = document.id;
  if (id == null) {
    throw ArgumentError('Dokumen harus memiliki id untuk diperbarui.');
  }

  final Database db = await DatabaseHelper.instance.database;
  await db.update(
    'documents',
    document.copyWith(updatedAt: DateTime.now()).toMap(),
    where: 'id = ?',
    whereArgs: [id],
  );
}
```

Tahap 15 — Konfirmasi Hapus Dokumen

Aksi hapus tidak langsung menghapus data. Aplikasi menampilkan dialog konfirmasi *Hapus Dokumen* dengan pesan peringatan dan tombol *Ya* / *Batal*. Setelah dikonfirmasi, data dihapus dari SQLite, SnackBar *Dokumen berhasil dihapus* ditampilkan, dan daftar dimuat ulang.



Gambar 3.15 Dialog konfirmasi hapus
Dialog hapus dengan tombol Batal dan Ya.

Method hapus dokumen — `lib/features/settings/settings_page.dart` (baris 65–94)

```
Future<void> _deleteDocument(Document document) async {
  final bool confirmed = await showConfirmDialog(
    context,
    title: 'Hapus Dokumen',
    message:
        'Yakin ingin menghapus "${document.title}"? '
        'Data yang dihapus tidak dapat dikembalikan.',
  );

  if (!confirmed || !mounted) return;

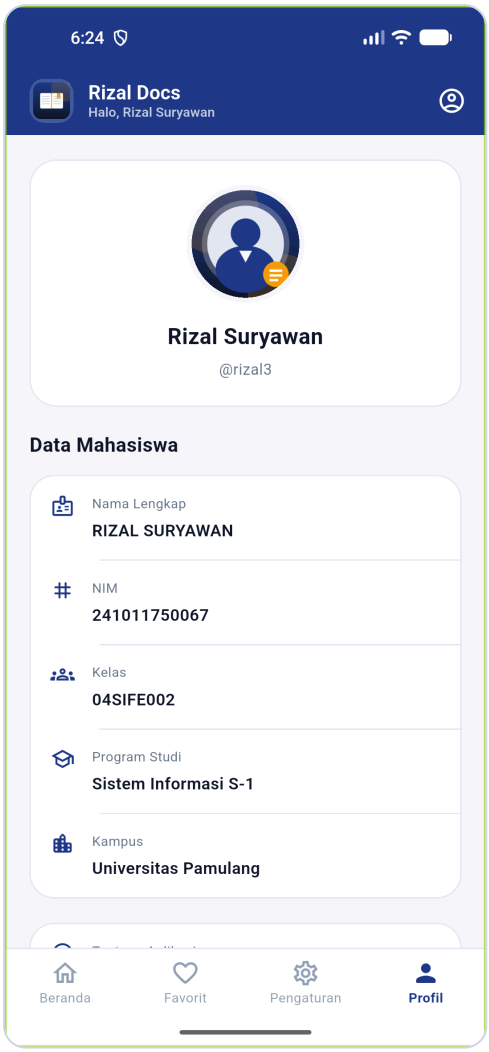
  try {
    final int? id = document.id;
    if (id == null) {
      throw StateError('Dokumen tidak memiliki id.');
    }

    await _repository.delete(id);
    if (!mounted) return;
    showAppSnackBar(context, 'Dokumen berhasil dihapus');
    await _loadDocuments();
  } catch (error) {
    if (!mounted) return;
    showAppDialog(
      context,
      title: 'Gagal Menghapus',
      message: 'Terjadi kesalahan saat menghapus dokumen: $error',
```

```
      );
    }
  }
```

Tahap 16 — Profil Mahasiswa

Tab Profil menampilkan foto profil, nama pengguna, username, serta data mahasiswa (nama, NIM, kelas, program studi, dan kampus). Halaman ini juga menyediakan tombol keluar yang kembali ke halaman login.



Gambar 3.16 Halaman profil
Identitas mahasiswa dan data akun yang sedang login.

Tahap 17 — Pengujian Aplikasi

Pengujian dilakukan pada tiga tingkat: analisis statis, unit/widget test, dan integration test (end-to-end) pada emulator Android. Integration test mencakup registrasi, login, pembukaan detail, penambahan favorit, pembukaan tab favorit, penambahan dokumen, perubahan dokumen, dan penghapusan dokumen.

Pengujian	Hasil
`flutter analyze`	No issues found
Unit test validator (6 kasus)	Lulus
Unit test model Document (3 kasus)	Lulus
Widget test splash dan validasi login (2 kasus)	Lulus
Integration test alur utama	Lulus

```
$ flutter analyze
No issues found! (ran in 1.8s)

$ flutter test
00:01 +11: All tests passed!

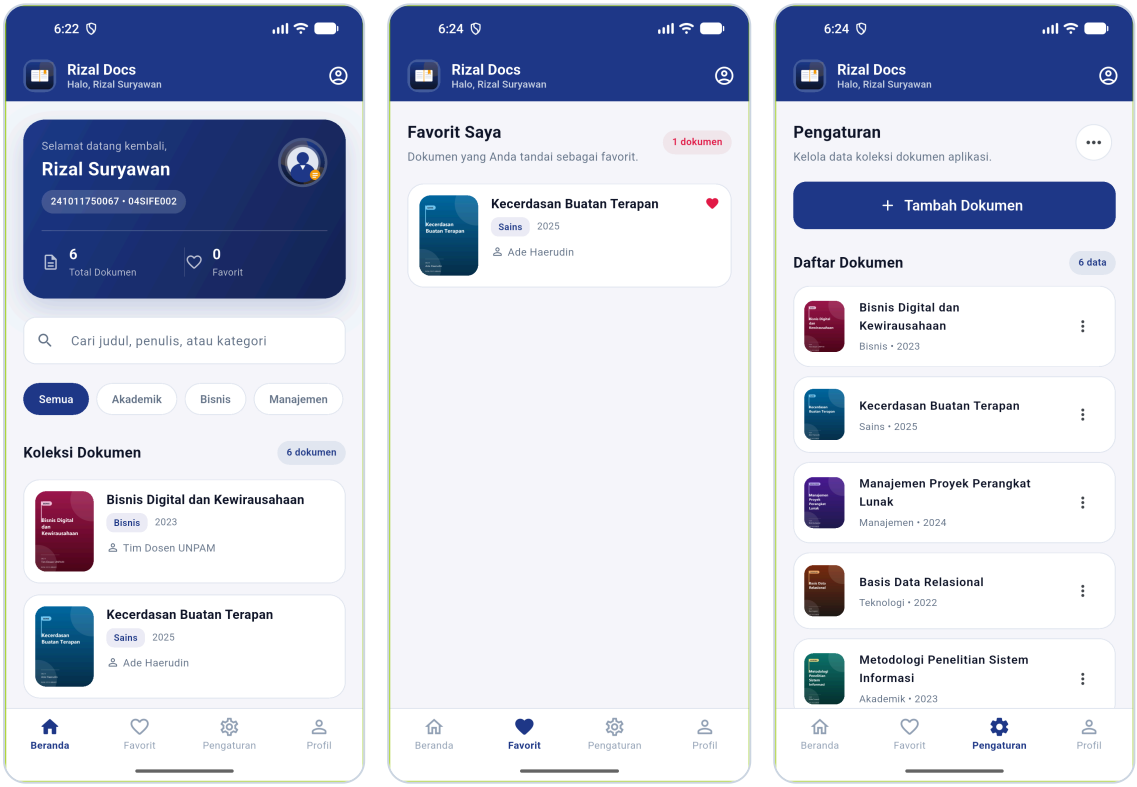
$ flutter test integration_test/app_test.dart -d emulator-5554
00:24 +1: All tests passed!
```

BAB 4 · HASIL AKHIR

4.1 Ringkasan Hasil

Soal	Hasil yang Dicapai	Status
1	Aplikasi bernama Rizal Docs dengan foto profil bertema dan seluruh gambar dari aset lokal. Struktur folder aset didokumentasikan pada laporan dan README.	Selesai
2	Registrasi dan login menggunakan Form, GlobalKey, TextFormField, validator, SQLite, SnackBar, dan AlertDialog.	Selesai
3	Dashboard StatefulWidget dengan AppBar, body scroll, BottomNavigationBar, ListView.builder, dan nama pengguna.	Selesai
4	Menu Pengaturan dengan CRUD, detail melalui Navigator.push, dialog konfirmasi favorit Ya/Batal, dan SnackBar.	Selesai

4.2 Galeri Hasil Akhir



Dashboard, tab Favorit, dan menu Pengaturan pada aplikasi versi akhir.

4.3 Informasi Build

Item	Keterangan
Nama package	`rizal_apps`
Application ID Android	`com.rizal.rizal_apps`
Label aplikasi	Rizal Docs
Jumlah file Dart (lib)	28 file, 4154 baris
Jumlah file pengujian	4 file, 320 baris
Perintah build	`flutter build apk --release`
Lokasi APK	`build/app/outputs/flutter-apk/app-release.apk`

BAB 5 · KESIMPULAN

Aplikasi Rizal Docs berhasil dibangun sesuai tema repositori/koleksi dokumen yang dipilih berdasarkan angka terakhir NIM. Seluruh ketentuan UTS terpenuhi: antarmuka login dengan foto profil bertema, autentikasi berbasis SQLite dengan validasi form, dashboard dengan header dan bottom navigation, halaman detail dengan aksi favorit berdialog konfirmasi, serta pengelolaan dokumen (create, read, update, delete) melalui menu Pengaturan.

Pengerjaan dilakukan secara bertahap dan setiap tahap diverifikasi dengan `flutter analyze` dan pengujian. Hasil akhir menunjukkan tidak ada issue pada analisis statis, sebelas unit/widget test lulus, dan integration test alur utama lulus pada emulator Android. Seluruh gambar menggunakan aset lokal sehingga aplikasi tetap dapat digunakan tanpa koneksi internet.

Beberapa hal yang dapat dikembangkan pada versi berikutnya: penyimpanan sesi login agar pengguna tidak perlu masuk ulang, hashing password sebelum disimpan, pencarian lanjutan, serta sinkronisasi data ke server. Untuk lingkup UTS, aplikasi sudah memenuhi seluruh kebutuhan fungsional dan ketentuan soal.

LAMPIRAN A · SOURCE CODE APLIKASI

Lampiran ini memuat seluruh source code aplikasi pada folder `lib` tanpa pemotongan. Setiap berkas diawali dengan judul berwarna biru yang menyebutkan path dan jumlah baris.

lib/app.dart (29 baris)

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'core/constants/app_constants.dart';
import 'core/theme/app_theme.dart';
import 'features/splash/splash_page.dart';

/// Root widget aplikasi Rizal Docs.
class RizalDocsApp extends StatelessWidget {
  const RizalDocsApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: AppConstants.appName,
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      // Header seluruh halaman berwarna biru tua, sehingga ikon status bar
      // dibuat terang agar tetap terbaca.
      builder: (BuildContext context, Widget? child) {
        return AnnotatedRegion<SystemUiOverlayStyle>(
          value: SystemUiOverlayStyle.light,
          child: child ?? const SizedBox.shrink(),
        );
      },
      home: const SplashPage(),
    );
  }
}
```

lib/core/constants/app_constants.dart (41 baris)

```
/// Konstanta global aplikasi: identitas mahasiswa, nama aplikasi, dan
/// konfigurasi database.
///
/// Nilai identitas dipakai pada halaman login, profil, dan header dashboard
/// sehingga cukup diubah di satu tempat.
class AppConstants {
  const AppConstants._();

  // Identitas aplikasi.
  static const String appName = 'Rizal Docs';
  static const String appTagline = 'Repositori & Koleksi Dokumen';
  static const String appVersion = '1.0.0';

  // Identitas mahasiswa (Soal 1 - identitas project).
  static const String studentName = 'RIZAL SURYAWAN';
  static const String studentNim = '241011750067';
  static const String studentClass = '04SIFE002';
  static const String studentProgram = 'Sistem Informasi S-1';
  static const String studentCampus = 'Universitas Pamulang';

  // Konfigurasi database SQLite.
  static const String databaseName = 'rizal_docs.db';
  static const int databaseVersion = 2;

  static const String usersTable = 'users';
  static const String documentsTable = 'documents';

  // Folder aset lokal.
```

```
  static const String profileImage = 'assets/images/profile/profile_rizal.png';
  static const String logoImage = 'assets/images/brand/logo_rizal_docs.png';

  /// Sampul dokumen yang dipakai sebagai data awal (seed) pada dashboard.
  static const List<String> coverImages = [
    'assets/images/covers/cover_01.png',
    'assets/images/covers/cover_02.png',
    'assets/images/covers/cover_03.png',
    'assets/images/covers/cover_04.png',
    'assets/images/covers/cover_05.png',
    'assets/images/covers/cover_06.png',
  ];
}
```

lib/core/database/database_helper.dart (109 baris)

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

import '../../data/document_seed.dart';
import '../../models/document.dart';
import '../constants/app_constants.dart';

/// Pengelola koneksi database SQLite aplikasi.
///
/// Menggunakan pola singleton agar hanya ada satu koneksi database selama
/// aplikasi berjalan.
class DatabaseHelper {
  DatabaseHelper._();

  static final DatabaseHelper instance = DatabaseHelper._();

  static Database? _database;

  Future<Database> get database async {
    return _database ??= await _openDatabase();
  }

  Future<Database> _openDatabase() async {
    final String databasePath = join(
      await getDatabasesPath(),
      AppConstants.databaseName,
    );

    return openDatabase(
      databasePath,
      version: AppConstants.databaseVersion,
      onConfigure: (Database db) async {
        await db.execute('PRAGMA foreign_keys = ON');
      },
      onCreate: _onCreate,
      onUpgrade: _onUpgrade,
    );
  }

  Future<void> _onCreate(Database db, int version) async {
    await _createUsersTable(db);
    await _createDocumentsTable(db);
    await _seedDocuments(db);
  }

  Future<void> _onUpgrade(Database db, int oldVersion, int newVersion) async {
    // Versi 2: menambahkan tabel dokumen beserta data awal dari aset lokal.
    if (oldVersion < 2) {
      await _createDocumentsTable(db);
      await _seedDocuments(db);
    }
  }

  Future<void> _createUsersTable(Database db) async {
    await db.execute('''
      CREATE TABLE ${AppConstants.usersTable} (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nama TEXT NOT NULL,
```

```
        username TEXT NOT NULL UNIQUE,
        password TEXT NOT NULL,
        created_at TEXT NOT NULL
      )
    ''');
  }

  Future<void> _createDocumentsTable(Database db) async {
    await db.execute('''
      CREATE TABLE ${AppConstants.documentsTable} (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        category TEXT NOT NULL,
        author TEXT NOT NULL,
        year INTEGER NOT NULL,
        description TEXT NOT NULL,
        image TEXT NOT NULL,
        is_favorite INTEGER NOT NULL DEFAULT 0,
        created_at TEXT NOT NULL,
        updated_at TEXT NOT NULL
      )
    ''');
  }

  Future<void> _seedDocuments(Database db) async {
    final int count =
        Sqflite.firstIntValue(
          await db.rawQuery(
            'SELECT COUNT(*) FROM ${AppConstants.documentsTable}',
          ),
        ) ??
        0;

    if (count > 0) return;

    final Batch batch = db.batch();
    for (final Document document in DocumentSeed.items) {
      batch.insert('documents', document.toMap()..remove('id'));
    }
    await batch.commit(noResult: true);
  }

  /// Menutup koneksi database. Dipakai oleh pengujian.
  Future<void> close() async {
    final Database? db = _database;
    if (db != null && db.isOpen) {
      await db.close();
    }
    _database = null;
  }
}
```

lib/core/theme/app_theme.dart (116 baris)

```
import 'package:flutter/material.dart';

/// Tema utama aplikasi Rizal Docs.
///
/// Palet warna diambil dari nuansa "koleksi dokumen": biru tua sebagai warna
/// utama dan amber sebagai aksen.
class AppTheme {
  const AppTheme._();

  static const Color primary = Color(0xFF1E3A8A);
  static const Color primaryDark = Color(0xFF172554);
  static const Color accent = Color(0xFFF59E0B);
  static const Color scaffold = Color(0xFFF5F7FB);
  static const Color textPrimary = Color(0xFF0F172A);
  static const Color textSecondary = Color(0xFF64748B);
  static const Color border = Color(0xFFE2E8F0);

  static ThemeData get light {
    final ColorScheme scheme = ColorScheme.fromSeed(
      seedColor: primary,
```

```
    ).copyWith(primary: primary, secondary: accent, surface: Colors.white);

    final BorderRadius radius = BorderRadius.circular(14);

    return ThemeData(
      useMaterial3: true,
      colorScheme: scheme,
      scaffoldBackgroundColor: scaffold,
      appBarTheme: const AppBarTheme(
        backgroundColor: primary,
        foregroundColor: Colors.white,
        elevation: 0,
        centerTitle: false,
        titleTextStyle: TextStyle(
          fontSize: 18,
          fontWeight: FontWeight.w600,
          color: Colors.white,
        ),
      ),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: Colors.white,
        contentPadding: const EdgeInsets.symmetric(
          horizontal: 16,
          vertical: 14,
        ),
        hintStyle: const TextStyle(color: textSecondary, fontSize: 14),
        labelStyle: const TextStyle(color: textSecondary),
        border: OutlineInputBorder(
          borderRadius: radius,
          borderSide: const BorderSide(color: border),
        ),
        enabledBorder: OutlineInputBorder(
          borderRadius: radius,
          borderSide: const BorderSide(color: border),
        ),
        focusedBorder: OutlineInputBorder(
          borderRadius: radius,
          borderSide: const BorderSide(color: primary, width: 1.6),
        ),
        errorBorder: OutlineInputBorder(
          borderRadius: radius,
          borderSide: BorderSide(color: scheme.error),
        ),
        focusedErrorBorder: OutlineInputBorder(
          borderRadius: radius,
          borderSide: BorderSide(color: scheme.error, width: 1.6),
        ),
      ),
      filledButtonTheme: FilledButtonThemeData(
        style: FilledButton.styleFrom(
          backgroundColor: primary,
          foregroundColor: Colors.white,
          minimumSize: const Size.fromHeight(52),
          shape: RoundedRectangleBorder(borderRadius: radius),
          textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.w600),
        ),
      ),
      outlinedButtonTheme: OutlinedButtonThemeData(
        style: OutlinedButton.styleFrom(
          foregroundColor: primary,
          minimumSize: const Size.fromHeight(50),
          side: const BorderSide(color: border),
          shape: RoundedRectangleBorder(borderRadius: radius),
          textStyle: const TextStyle(fontSize: 15, fontWeight: FontWeight.w600),
        ),
      ),
      textButtonTheme: TextButtonThemeData(
        style: TextButton.styleFrom(
          foregroundColor: primary,
          textStyle: const TextStyle(fontWeight: FontWeight.w600),
        ),
      ),
      snackBarTheme: SnackBarThemeData(
        backgroundColor: const Color(0xFF0F172A),
        behavior: SnackBarBehavior.floating,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
```

```
        contentTextStyle: const TextStyle(color: Colors.white),
      ),
      dividerTheme: const DividerThemeData(
        color: border,
        thickness: 1,
        space: 1,
      ),
      textTheme: const TextTheme(
        headlineSmall: TextStyle(
          color: textPrimary,
          fontWeight: FontWeight.w700,
        ),
        titleMedium: TextStyle(color: textPrimary, fontWeight: FontWeight.w600),
        bodyMedium: TextStyle(color: textPrimary, height: 1.5),
        bodySmall: TextStyle(color: textSecondary, height: 1.4),
      ),
    );
  }
}
```

lib/core/utils/date_formatter.dart (7 baris)

```
import 'package:intl/intl.dart';

/// Memformat tanggal menjadi teks berbahasa Indonesia,
/// contoh: `18 Juni 2024`.
String formatTanggalPanjang(DateTime date) {
  return DateFormat('d MMMM y', 'id_ID').format(date);
}
```

lib/core/utils/validators.dart (66 baris)

```
/// Kumpulan validator form yang dipakai pada halaman login dan registrasi.
///
/// Dipisahkan dari widget agar mudah diuji dan dipakai ulang.
class AppValidators {
  const AppValidators._();

  static String? nama(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Nama lengkap tidak boleh kosong';
    }
    if (value.trim().length < 3) {
      return 'Nama lengkap minimal 3 karakter';
    }
    return null;
  }

  static String? username(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Username tidak boleh kosong';
    }
    return null;
  }

  static String? password(String? value) {
    if (value == null || value.isEmpty) {
      return 'Password tidak boleh kosong';
    }
    if (value.length < 6) {
      return 'Password minimal 6 karakter';
    }
    return null;
  }

  static String? konfirmasiPassword(String? value, String password) {
    if (value == null || value.isEmpty) {
      return 'Konfirmasi password tidak boleh kosong';
    }
    if (value != password) {
```

```
      return 'Konfirmasi password tidak sesuai';
    }
    return null;
  }

  /// Validator umum untuk field teks yang wajib diisi.
  static String? requiredField(String? value, String label) {
    if (value == null || value.trim().isEmpty) {
      return '$label tidak boleh kosong';
    }
    return null;
  }

  /// Validator tahun terbit dokumen (1900 - 2100).
  static String? tahun(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Tahun tidak boleh kosong';
    }
    final int? year = int.tryParse(value.trim());
    if (year == null) {
      return 'Tahun harus berupa angka';
    }
    if (year < 1900 || year > 2100) {
      return 'Tahun harus di antara 1900 dan 2100';
    }
    return null;
  }
}
```

lib/core/widgets/app_feedback.dart (103 baris)

```
import 'package:flutter/material.dart';

import '../theme/app_theme.dart';

/// Menampilkan SnackBar singkat di bagian bawah layar.
void showAppSnackBar(
  BuildContext context,
  String message, {
  bool isError = false,
}) {
  final ScaffoldMessengerState messenger = ScaffoldMessenger.of(context);
  messenger
    ..hideCurrentSnackBar()
    ..showSnackBar(
      SnackBar(
        content: Row(
          children: [
            Icon(
              isError
                  ? Icons.error_outline_rounded
                  : Icons.check_circle_outline,
              color: isError
                  ? const Color(0xFFFCA5A5)
                  : const Color(0xFF6EE7B7),
              size: 20,
            ),
            const SizedBox(width: 10),
            Expanded(child: Text(message)),
          ],
        ),
      ),
    );
}

/// Menampilkan AlertDialog dengan satu tombol OK.
Future<void> showAppDialog(
  BuildContext context, {
  required String title,
  required String message,
  String buttonLabel = 'OK',
}) {
  return showDialog<void>(
    context: context,
```

```
    builder: (BuildContext dialogContext) {
      return AlertDialog(
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),
        title: Text(
          title,
          style: const TextStyle(fontWeight: FontWeight.w700, fontSize: 18),
        ),
        content: Text(message),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(dialogContext).pop(),
            child: Text(buttonLabel),
          ),
        ],
      );
    },
  );
}

/// Menampilkan AlertDialog konfirmasi dengan pilihan Ya/Batal.
Future<bool> showConfirmDialog(
  BuildContext context, {
  required String title,
  required String message,
  String confirmLabel = 'Ya',
  String cancelLabel = 'Batal',
}) async {
  final bool? result = await showDialog<bool>(
    context: context,
    builder: (BuildContext dialogContext) {
      return AlertDialog(
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),
        title: Text(
          title,
          style: const TextStyle(fontWeight: FontWeight.w700, fontSize: 18),
        ),
        content: Text(message),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(dialogContext).pop(false),
            child: Text(
              cancelLabel,
              style: const TextStyle(color: AppTheme.textSecondary),
            ),
          ),
          FilledButton(
            onPressed: () => Navigator.of(dialogContext).pop(true),
            style: FilledButton.styleFrom(
              minimumSize: const Size(0, 40),
              padding: const EdgeInsets.symmetric(horizontal: 20),
            ),
            child: Text(confirmLabel),
          ),
        ],
      );
    },
  );

  return result ?? false;
}
```

lib/core/widgets/app_text_field.dart (71 baris)

```
import 'package:flutter/material.dart';

/// Field teks standar aplikasi dengan style yang konsisten.
class AppTextField extends StatelessWidget {
  const AppTextField({
    super.key,
    required this.label,
    required this.controller,
    this.hint,
    this.prefixIcon,
    this.suffixIcon,
```

```
    this.obscureText = false,
    this.validator,
    this.keyboardType,
    this.textInputAction,
    this.autofillHints,
    this.onFieldSubmitted,
    this.readOnly = false,
    this.maxLines = 1,
  });

  final String label;
  final String? hint;
  final TextEditingController controller;
  final IconData? prefixIcon;
  final Widget? suffixIcon;
  final bool obscureText;
  final FormFieldValidator<String>? validator;
  final TextInputType? keyboardType;
  final TextInputAction? textInputAction;
  final Iterable<String>? autofillHints;
  final ValueChanged<String>? onFieldSubmitted;
  final bool readOnly;
  final int maxLines;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          label,
          style: const TextStyle(
            fontSize: 13,
            fontWeight: FontWeight.w600,
            color: Color(0xFF334155),
          ),
        ),
        const SizedBox(height: 8),
        TextFormField(
          controller: controller,
          obscureText: obscureText,
          validator: validator,
          keyboardType: keyboardType,
          textInputAction: textInputAction,
          autofillHints: autofillHints,
          onFieldSubmitted: onFieldSubmitted,
          readOnly: readOnly,
          maxLines: maxLines,
          decoration: InputDecoration(
            hintText: hint,
            prefixIcon: prefixIcon == null
                ? null
                : Icon(prefixIcon, size: 20, color: const Color(0xFF64748B)),
            suffixIcon: suffixIcon,
          ),
        ),
      ],
    );
  }
}
```

lib/core/widgets/primary_button.dart (44 baris)

```
import 'package:flutter/material.dart';

/// Tombol utama aplikasi dengan indikator loading.
class PrimaryButton extends StatelessWidget {
  const PrimaryButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.isLoading = false,
    this.icon,
  });
```

```
  final String label;
  final VoidCallback? onPressed;
  final bool isLoading;
  final IconData? icon;

  @override
  Widget build(BuildContext context) {
    return FilledButton(
      onPressed: isLoading ? null : onPressed,
      child: isLoading
          ? const SizedBox(
              width: 22,
              height: 22,
              child: CircularProgressIndicator(
                strokeWidth: 2.4,
                valueColor: AlwaysStoppedAnimation<Color>(Colors.white),
              ),
            )
          : Row(
              mainAxisAlignment: MainAxisAlignment.center,
              mainAxisSize: MainAxisSize.min,
              children: [
                if (icon != null) ...[
                  Icon(icon, size: 20),
                  const SizedBox(width: 8),
                ],
                Text(label),
              ],
            ),
    );
  }
}
```

lib/core/widgets/section_header.dart (45 baris)

```
import 'package:flutter/material.dart';

import '../theme/app_theme.dart';

/// Judul bagian pada halaman yang dapat di-scroll.
class SectionHeader extends StatelessWidget {
  const SectionHeader({
    super.key,
    required this.title,
    this.actionLabel,
    this.onActionPressed,
  });

  final String title;
  final String? actionLabel;
  final VoidCallback? onActionPressed;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Expanded(
          child: Text(
            title,
            style: const TextStyle(
              fontSize: 16,
              fontWeight: FontWeight.w700,
              color: AppTheme.textPrimary,
            ),
          ),
        ),
        if (actionLabel != null)
          TextButton(
            onPressed: onActionPressed,
            style: TextButton.styleFrom(
              padding: const EdgeInsets.symmetric(horizontal: 8),
              minimumSize: const Size(0, 32),
              tapTargetSize: MaterialTapTargetSize.shrinkWrap,
```

```
            ),
            child: Text(actionLabel!, style: const TextStyle(fontSize: 12)),
          ),
      ],
    );
  }
}
```

lib/data/document_seed.dart (88 baris)

```
import '../models/document.dart';

/// Data awal dokumen yang disimpan ke SQLite saat aplikasi pertama kali
/// dijalankan. Gambar diambil dari folder `assets/images/covers/`.
class DocumentSeed {
  const DocumentSeed._();

  static List<Document> get items => [
    Document(
      title: 'Dasar-Dasar Pemrograman Mobile',
      category: 'Teknologi',
      author: 'Samso Supriyatna',
      year: 2024,
      description:
          'Panduan pengantar pengembangan aplikasi mobile menggunakan '
          'Flutter. Membahas struktur project, widget dasar, layout, '
          'serta pengelolaan state untuk pemula.',
      image: 'assets/images/covers/cover_01.png',
      createdAt: DateTime(2024, 1, 10),
      updatedAt: DateTime(2024, 1, 10),
    ),
    Document(
      title: 'Metodologi Penelitian Sistem Informasi',
      category: 'Akademik',
      author: 'Meta Susanti',
      year: 2023,
      description:
          'Buku ajar yang menjelaskan tahapan penelitian sistem informasi, '
          'mulai dari perumusan masalah, pengumpulan data, hingga penyusunan '
          'laporan penelitian yang baik.',
      image: 'assets/images/covers/cover_02.png',
      createdAt: DateTime(2024, 2, 14),
      updatedAt: DateTime(2024, 2, 14),
    ),
    Document(
      title: 'Basis Data Relasional',
      category: 'Teknologi',
      author: 'Gusmayeni',
      year: 2022,
      description:
          'Membahas konsep basis data relasional, normalisasi, perancangan '
          'tabel, serta penerapan query SQL untuk kebutuhan aplikasi '
          'berbasis data.',
      image: 'assets/images/covers/cover_03.png',
      createdAt: DateTime(2024, 3, 8),
      updatedAt: DateTime(2024, 3, 8),
    ),
    Document(
      title: 'Manajemen Proyek Perangkat Lunak',
      category: 'Manajemen',
      author: 'Rizal Suryawan',
      year: 2024,
      description:
          'Ringkasan praktik manajemen proyek perangkat lunak: penyusunan '
          'kebutuhan, estimasi waktu, pembagian tugas tim, serta pemantauan '
          'progres pengembangan.',
      image: 'assets/images/covers/cover_04.png',
      createdAt: DateTime(2024, 4, 21),
      updatedAt: DateTime(2024, 4, 21),
    ),
    Document(
      title: 'Kecerdasan Buatan Terapan',
      category: 'Sains',
      author: 'Ade Haerudin',
```

```
      year: 2025,
      description:
          'Pengantar penerapan kecerdasan buatan pada aplikasi sehari-hari, '
          'meliputi pengenalan pola, rekomendasi, dan pemanfaatan model '
          'machine learning secara sederhana.',
      image: 'assets/images/covers/cover_05.png',
      createdAt: DateTime(2024, 5, 30),
      updatedAt: DateTime(2024, 5, 30),
    ),
    Document(
      title: 'Bisnis Digital dan Kewirausahaan',
      category: 'Bisnis',
      author: 'Tim Dosen UNPAM',
      year: 2023,
      description:
          'Materi tentang peluang usaha digital, penyusunan model bisnis, '
          'strategi pemasaran daring, dan pengelolaan keuangan usaha kecil '
          'berbasis teknologi.',
      image: 'assets/images/covers/cover_06.png',
      createdAt: DateTime(2024, 6, 18),
      updatedAt: DateTime(2024, 6, 18),
    ),
  ];
}
```

lib/features/auth/login_page.dart (189 baris)

```
import 'package:flutter/material.dart';

import '../../core/theme/app_theme.dart';
import '../../core/utils/validators.dart';
import '../../core/widgets/app_feedback.dart';
import '../../core/widgets/app_text_field.dart';
import '../../core/widgets/primary_button.dart';
import '../../models/user.dart';
import '../../repositories/auth_repository.dart';
import '../dashboard/dashboard_page.dart';
import 'register_page.dart';
import 'widgets/auth_scaffold.dart';

/// Halaman login aplikasi.
class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final TextEditingController _usernameController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();

  bool _obscurePassword = true;
  bool _isLoading = false;

  @override
  void dispose() {
    _usernameController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  /// Memvalidasi form lalu mencocokkan data dengan database SQLite.
  Future<void> actionLogin() async {
    FocusScope.of(context).unfocus();

    if (!(_formKey.currentState?.validate() ?? false)) return;

    setState(() => _isLoading = true);

    try {
      final User? user = await AuthRepository.instance.login(
        _usernameController.text,
```

```
        _passwordController.text,
      );

      if (!mounted) return;

      if (user == null) {
        await showAppDialog(
          context,
          title: 'Login Gagal',
          message: 'Username atau password salah',
        );
        return;
      }

      showAppSnackBar(context, 'Login berhasil');
      Navigator.of(context).pushReplacement(
        MaterialPageRoute<void>(
          builder: (BuildContext context) => DashboardPage(user: user),
        ),
      );
    } catch (error) {
      if (!mounted) return;
      showAppDialog(
        context,
        title: 'Terjadi Kesalahan',
        message: 'Tidak dapat mengakses database: $error',
      );
    } finally {
      if (mounted) setState(() => _isLoading = false);
    }
  }

  Future<void> _openRegisterPage() async {
    final String? registeredUsername = await Navigator.of(context).push<String>(
      MaterialPageRoute<String>(
        builder: (BuildContext context) => const RegisterPage(),
      ),
    );

    if (!mounted) return;
    if (registeredUsername != null && registeredUsername.isNotEmpty) {
      _usernameController.text = registeredUsername;
      _passwordController.clear();
    }
  }

  @override
  Widget build(BuildContext context) {
    return AuthScaffold(
      children: [
        AuthCard(
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const Text(
                  'Masuk ke Akun',
                  style: TextStyle(
                    fontSize: 18,
                    fontWeight: FontWeight.w700,
                    color: AppTheme.textPrimary,
                  ),
                ),
                const SizedBox(height: 6),
                const Text(
                  'Silakan masuk untuk mulai mengelola koleksi dokumen Anda.',
                  style: TextStyle(fontSize: 12.5, color: Color(0xFF64748B)),
                ),
                const SizedBox(height: 20),
                AppTextField(
                  label: 'Username',
                  hint: 'Masukkan username',
                  controller: _usernameController,
                  prefixIcon: Icons.person_outline_rounded,
                  validator: AppValidators.username,
                  keyboardType: TextInputType.text,
```

```
                  textInputAction: TextInputAction.next,
                  autofillHints: const [AutofillHints.username],
                ),
                const SizedBox(height: 16),
                AppTextField(
                  label: 'Password',
                  hint: 'Minimal 6 karakter',
                  controller: _passwordController,
                  prefixIcon: Icons.lock_outline_rounded,
                  obscureText: _obscurePassword,
                  validator: AppValidators.password,
                  textInputAction: TextInputAction.done,
                  autofillHints: const [AutofillHints.password],
                  onFieldSubmitted: (_) => actionLogin(),
                  suffixIcon: IconButton(
                    onPressed: () {
                      setState(() => _obscurePassword = !_obscurePassword);
                    },
                    icon: Icon(
                      _obscurePassword
                          ? Icons.visibility_off_outlined
                          : Icons.visibility_outlined,
                      size: 20,
                      color: const Color(0xFF64748B),
                    ),
                    tooltip: _obscurePassword
                        ? 'Tampilkan password'
                        : 'Sembunyikan password',
                  ),
                ),
                const SizedBox(height: 22),
                PrimaryButton(
                  label: 'Masuk',
                  icon: Icons.login_rounded,
                  isLoading: _isLoading,
                  onPressed: actionLogin,
                ),
              ],
            ),
          ),
        ),
        const SizedBox(height: 18),
        Row(
          children: [
            const Expanded(child: Divider()),
            Padding(
              padding: const EdgeInsets.symmetric(horizontal: 12),
              child: Text(
                'Belum punya akun?',
                style: TextStyle(fontSize: 12, color: Colors.grey.shade600),
              ),
            ),
            const Expanded(child: Divider()),
          ],
        ),
        const SizedBox(height: 14),
        OutlinedButton.icon(
          onPressed: _isLoading ? null : _openRegisterPage,
          icon: const Icon(Icons.person_add_alt_1_rounded, size: 18),
          label: const Text('Registrasi Akun Baru'),
        ),
      ],
    );
  }
}
```

lib/features/auth/register_page.dart (182 baris)

```
import 'package:flutter/material.dart';

import '../../core/utils/validators.dart';
import '../../core/widgets/app_feedback.dart';
import '../../core/widgets/app_text_field.dart';
import '../../core/widgets/primary_button.dart';
```

```dart
import '../../models/user.dart';
import '../../repositories/auth_repository.dart';
import 'widgets/auth_scaffold.dart';

/// Halaman registrasi pengguna baru.
class RegisterPage extends StatefulWidget {
  const RegisterPage({super.key});

  @override
  State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final TextEditingController _namaController = TextEditingController();
  final TextEditingController _usernameController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final TextEditingController _konfirmasiController = TextEditingController();

  bool _obscurePassword = true;
  bool _isLoading = false;

  @override
  void dispose() {
    _namaController.dispose();
    _usernameController.dispose();
    _passwordController.dispose();
    _konfirmasiController.dispose();
    super.dispose();
  }

  /// Memvalidasi form lalu menyimpan pengguna baru ke database SQLite.
  Future<void> actionRegistrasi() async {
    FocusScope.of(context).unfocus();

    if (!(_formKey.currentState?.validate() ?? false)) return;

    setState(() => _isLoading = true);

    try {
      final String username = _usernameController.text.trim().toLowerCase();
      final bool isTaken = await AuthRepository.instance.isUsernameTaken(
        username,
      );

      if (!mounted) return;

      if (isTaken) {
        await showAppDialog(
          context,
          title: 'Registrasi Gagal',
          message: 'Username sudah digunakan, silakan pilih username lain.',
        );
        return;
      }

      final User user = await AuthRepository.instance.register(
        nama: _namaController.text,
        username: username,
        password: _passwordController.text,
      );

      if (!mounted) return;
      showAppSnackBar(context, 'Registrasi berhasil, silakan login');
      Navigator.of(context).pop(user.username);
    } catch (error) {
      if (!mounted) return;
      showAppDialog(
        context,
        title: 'Terjadi Kesalahan',
        message: 'Tidak dapat menyimpan data: $error',
      );
    } finally {
      if (mounted) setState(() => _isLoading = false);
    }
  }
```

```
@override
Widget build(BuildContext context) {
  return AuthScaffold(
    showBackButton: true,
    headerTitle: 'Buat Akun Baru',
    children: [
      AuthCard(
        child: Form(
          key: _formKey,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text(
                'Registrasi Pengguna',
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.w700),
              ),
              const SizedBox(height: 6),
              const Text(
                'Lengkapi data di bawah untuk membuat akun Rizal Docs.',
                style: TextStyle(fontSize: 12.5, color: Color(0xFF64748B)),
              ),
              const SizedBox(height: 20),
              AppTextField(
                label: 'Nama Lengkap',
                hint: 'Contoh: Rizal Suryawan',
                controller: _namaController,
                prefixIcon: Icons.badge_outlined,
                validator: AppValidators.nama,
                keyboardType: TextInputType.name,
                textInputAction: TextInputAction.next,
                autofillHints: const [AutofillHints.name],
              ),
              const SizedBox(height: 16),
              AppTextField(
                label: 'Username',
                hint: 'Username untuk login',
                controller: _usernameController,
                prefixIcon: Icons.person_outline_rounded,
                validator: AppValidators.username,
                keyboardType: TextInputType.text,
                textInputAction: TextInputAction.next,
                autofillHints: const [AutofillHints.newUsername],
              ),
              const SizedBox(height: 16),
              AppTextField(
                label: 'Password',
                hint: 'Minimal 6 karakter',
                controller: _passwordController,
                prefixIcon: Icons.lock_outline_rounded,
                obscureText: _obscurePassword,
                validator: AppValidators.password,
                textInputAction: TextInputAction.next,
                autofillHints: const [AutofillHints.newPassword],
              ),
              const SizedBox(height: 16),
              AppTextField(
                label: 'Konfirmasi Password',
                hint: 'Ulangi password',
                controller: _konfirmasiController,
                prefixIcon: Icons.lock_reset_rounded,
                obscureText: _obscurePassword,
                validator: (String? value) =>
                    AppValidators.konfirmasiPassword(
                      value,
                      _passwordController.text,
                    ),
                textInputAction: TextInputAction.done,
                onFieldSubmitted: (_) => actionRegistrasi(),
                suffixIcon: IconButton(
                  onPressed: () {
                    setState(() => _obscurePassword = !_obscurePassword);
                  },
                  icon: Icon(
                    _obscurePassword
                        ? Icons.visibility_off_outlined
                        : Icons.visibility_outlined,
                    size: 20,
```

```
                    color: const Color(0xFF64748B),
                  ),
                  tooltip: _obscurePassword
                      ? 'Tampilkan password'
                      : 'Sembunyikan password',
                ),
              ),
              const SizedBox(height: 22),
              PrimaryButton(
                label: 'Daftar',
                icon: Icons.person_add_alt_1_rounded,
                isLoading: _isLoading,
                onPressed: actionRegistrasi,
              ),
            ],
          ),
        ),
      ),
    ],
  );
 }
}
```

lib/features/auth/widgets/auth_scaffold.dart (162 baris)

```
import 'package:flutter/material.dart';

import '../../../core/constants/app_constants.dart';
import '../../../core/theme/app_theme.dart';

/// Kerangka halaman autentikasi (login & registrasi).
///
/// Bagian atas berisi branding dan foto profil bertema repositori dokumen,
/// sedangkan bagian bawah berisi form yang dapat di-scroll.
class AuthScaffold extends StatelessWidget {
  const AuthScaffold({
    super.key,
    required this.children,
    this.showBackButton = false,
    this.headerTitle,
  });

  final List<Widget> children;
  final bool showBackButton;
  final String? headerTitle;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppTheme.scaffold,
      body: Column(
        children: [
          Container(
            width: double.infinity,
            decoration: const BoxDecoration(
              gradient: LinearGradient(
                begin: Alignment.topCenter,
                end: Alignment.bottomCenter,
                colors: [AppTheme.primary, AppTheme.primaryDark],
              ),
              borderRadius: BorderRadius.only(
                bottomLeft: Radius.circular(32),
                bottomRight: Radius.circular(32),
              ),
            ),
            child: SafeArea(
              bottom: false,
              child: Stack(
                children: [
                  if (showBackButton)
                    Positioned(
                      left: 8,
                      top: 4,
                      child: IconButton(
```

```
                    onPressed: () => Navigator.of(context).maybePop(),
                    icon: const Icon(
                      Icons.arrow_back_rounded,
                      color: Colors.white,
                    ),
                    tooltip: 'Kembali',
                  ),
                ),
              SizedBox(
                width: double.infinity,
                child: Padding(
                  padding: EdgeInsets.fromLTRB(
                    24,
                    showBackButton ? 52 : 32,
                    24,
                    showBackButton ? 26 : 36,
                  ),
                  child: Column(
                    children: [
                      Container(
                        width: 104,
                        height: 104,
                        padding: const EdgeInsets.all(4),
                        decoration: BoxDecoration(
                          shape: BoxShape.circle,
                          color: Colors.white.withValues(alpha: 0.18),
                        ),
                        child: ClipOval(
                          child: Image.asset(
                            AppConstants.profileImage,
                            fit: BoxFit.cover,
                          ),
                        ),
                      ),
                      const SizedBox(height: 16),
                      Text(
                        headerTitle ?? AppConstants.appName,
                        textAlign: TextAlign.center,
                        style: const TextStyle(
                          color: Colors.white,
                          fontSize: 24,
                          fontWeight: FontWeight.w700,
                          letterSpacing: 0.3,
                        ),
                      ),
                      const SizedBox(height: 6),
                      Text(
                        AppConstants.appTagline,
                        textAlign: TextAlign.center,
                        style: TextStyle(
                          color: Colors.white.withValues(alpha: 0.75),
                          fontSize: 13,
                        ),
                      ),
                    ],
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
      Expanded(
        child: SingleChildScrollView(
          padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
          child: Column(
            children: [
              ...children,
              const SizedBox(height: 24),
              Text(
                '${AppConstants.studentName} • ${AppConstants.studentNim} • ${AppConstants.studentClass}',
                textAlign: TextAlign.center,
                style: const TextStyle(
                  fontSize: 11,
                  color: AppTheme.textSecondary,
                ),
              ),
```

```
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}

/// Kartu putih pembungkus form autentikasi.
class AuthCard extends StatelessWidget {
  const AuthCard({super.key, required this.child});

  final Widget child;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(24),
        boxShadow: const [
          BoxShadow(
            color: Color(0x0F0F172A),
            blurRadius: 24,
            offset: Offset(0, 12),
          ),
        ],
      ),
      child: child,
    );
  }
}
```

lib/features/dashboard/dashboard_page.dart (146 baris)

```
import 'package:flutter/material.dart';

import '../../core/constants/app_constants.dart';
import '../../core/theme/app_theme.dart';
import '../../models/user.dart';
import '../favorites/favorites_page.dart';
import '../profile/profile_page.dart';
import '../settings/settings_page.dart';
import 'widgets/home_tab.dart';

/// Halaman utama aplikasi setelah login.
///
/// Struktur halaman terdiri dari header (AppBar), body yang dapat di-scroll,
/// dan footer navigasi (BottomNavigationBar). Halaman ini menggunakan
/// [StatefulWidget] karena menyimpan status tab yang sedang aktif.
class DashboardPage extends StatefulWidget {
  const DashboardPage({super.key, required this.user});

  final User user;

  @override
  State<DashboardPage> createState() => _DashboardPageState();
}

class _DashboardPageState extends State<DashboardPage> {
  int _selectedIndex = 0;

  void _onDestinationSelected(int index) {
    if (index == _selectedIndex) return;
    setState(() => _selectedIndex = index);
  }

  Widget _buildBody() {
    switch (_selectedIndex) {
      case 1:
```

```
      return const FavoritesPage();
    case 2:
      return const SettingsPage();
    case 3:
      return ProfilePage(user: widget.user);
    default:
      return HomeTab(user: widget.user, onNavigate: _onDestinationSelected);
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      automaticallyImplyLeading: false,
      titleSpacing: 20,
      title: Row(
        children: [
          Container(
            width: 36,
            height: 36,
            padding: const EdgeInsets.all(3),
            decoration: BoxDecoration(
              color: Colors.white.withValues(alpha: 0.16),
              borderRadius: BorderRadius.circular(12),
            ),
            child: Image.asset(AppConstants.logoImage),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              mainAxisSize: MainAxisSize.min,
              children: [
                const Text(
                  AppConstants.appName,
                  style: TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w700,
                    color: Colors.white,
                  ),
                ),
                Text(
                  'Halo, ${widget.user.nama}',
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: TextStyle(
                    fontSize: 11,
                    color: Colors.white.withValues(alpha: 0.75),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
      actions: [
        IconButton(
          onPressed: () => _onDestinationSelected(3),
          icon: const Icon(Icons.account_circle_outlined),
          tooltip: 'Profil',
        ),
        const SizedBox(width: 4),
      ],
    ),
    body: SafeArea(child: _buildBody()),
    bottomNavigationBar: Container(
      decoration: const BoxDecoration(
        color: Colors.white,
        border: Border(top: BorderSide(color: AppTheme.border)),
      ),
      child: BottomNavigationBar(
        currentIndex: _selectedIndex,
        onTap: _onDestinationSelected,
        type: BottomNavigationBarType.fixed,
        backgroundColor: Colors.white,
        elevation: 0,
```

```
          selectedItemColor: AppTheme.primary,
          unselectedItemColor: const Color(0xFF94A3B8),
          selectedLabelStyle: const TextStyle(
            fontSize: 11,
            fontWeight: FontWeight.w700,
          ),
          unselectedLabelStyle: const TextStyle(fontSize: 11),
          items: const [
            BottomNavigationBarItem(
              icon: Icon(Icons.home_outlined),
              activeIcon: Icon(Icons.home_rounded),
              label: 'Beranda',
            ),
            BottomNavigationBarItem(
              icon: Icon(Icons.favorite_border_rounded),
              activeIcon: Icon(Icons.favorite_rounded),
              label: 'Favorit',
            ),
            BottomNavigationBarItem(
              icon: Icon(Icons.settings_outlined),
              activeIcon: Icon(Icons.settings_rounded),
              label: 'Pengaturan',
            ),
            BottomNavigationBarItem(
              icon: Icon(Icons.person_outline_rounded),
              activeIcon: Icon(Icons.person_rounded),
              label: 'Profil',
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/features/dashboard/widgets/home_tab.dart (510 baris)

```
import 'package:flutter/material.dart';

import '../../../core/constants/app_constants.dart';
import '../../../core/theme/app_theme.dart';
import '../../../models/document.dart';
import '../../../models/user.dart';
import '../../../repositories/document_repository.dart';
import '../../documents/document_detail_page.dart';
import '../../documents/widgets/document_card.dart';

/// Konten utama tab Beranda pada dashboard.
///
/// Menampilkan sapaan pengguna, ringkasan koleksi, pencarian, filter kategori,
/// dan daftar dokumen yang dibaca dari database SQLite. Seluruh konten
/// dibangun dengan [ListView.builder] sehingga halaman dapat di-scroll dan
/// data ditampilkan secara dinamis.
class HomeTab extends StatefulWidget {
  const HomeTab({super.key, required this.user, required this.onNavigate});

  final User user;
  final ValueChanged<int> onNavigate;

  @override
  State<HomeTab> createState() => _HomeTabState();
}

class _HomeTabState extends State<HomeTab> {
  static const String _allCategory = 'Semua';

  final TextEditingController _searchController = TextEditingController();
  final DocumentRepository _repository = DocumentRepository.instance;

  List<Document> _documents = [];
  String _selectedCategory = _allCategory;
  String _searchQuery = '';
  bool _isLoading = true;
  String? _errorMessage;
```

```
@override
void initState() {
  super.initState();
  _loadDocuments();
}

@override
void dispose() {
  _searchController.dispose();
  super.dispose();
}

Future<void> _loadDocuments() async {
  setState(() {
    _isLoading = true;
    _errorMessage = null;
  });

  try {
    final List<Document> documents = await _repository.getAll();
    if (!mounted) return;
    setState(() {
      _documents = documents;
      _isLoading = false;
    });
  } catch (error) {
    if (!mounted) return;
    setState(() {
      _errorMessage = 'Gagal memuat dokumen: $error';
      _isLoading = false;
    });
  }
}

Future<void> _openDetail(Document document) async {
  await Navigator.of(context).push(
    MaterialPageRoute<void>(
      builder: (BuildContext context) =>
          DocumentDetailPage(document: document),
    ),
  );

  if (!mounted) return;
  // Memuat ulang data agar status favorit pada daftar tetap sinkron.
  await _loadDocuments();
}

List<String> get _categories {
  final Set<String> categories = <String>{
    for (final Document document in _documents) document.category,
  };
  final List<String> sorted = categories.toList()..sort();
  return <String>[_allCategory, ...sorted];
}

List<Document> get _filteredDocuments {
  final String query = _searchQuery.trim().toLowerCase();

  return _documents.where((Document document) {
    final bool matchesCategory =
        _selectedCategory == _allCategory ||
        document.category == _selectedCategory;
    final bool matchesQuery =
        query.isEmpty ||
        document.title.toLowerCase().contains(query) ||
        document.author.toLowerCase().contains(query) ||
        document.category.toLowerCase().contains(query);

    return matchesCategory && matchesQuery;
  }).toList();
}

@override
Widget build(BuildContext context) {
  final List<Document> documents = _filteredDocuments;
```

```
  if (_isLoading) {
    return const Center(child: CircularProgressIndicator());
  }

  return RefreshIndicator(
    onRefresh: _loadDocuments,
    color: AppTheme.primary,
    child: ListView.builder(
      padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
      physics: const AlwaysScrollableScrollPhysics(),
      itemCount: documents.length + 1,
      itemBuilder: (BuildContext context, int index) {
        if (index == 0) {
          return _buildHeader(documents);
        }

        final Document document = documents[index - 1];
        return Padding(
          padding: const EdgeInsets.only(bottom: 12),
          child: DocumentCard(
            document: document,
            onTap: () => _openDetail(document),
          ),
        );
      },
    ),
  );
}

Widget _buildHeader(List<Document> filteredDocuments) {
  final int totalFavorites = _documents
      .where((Document document) => document.isFavorite)
      .length;

  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      _WelcomeCard(
        user: widget.user,
        totalDocuments: _documents.length,
        totalFavorites: totalFavorites,
      ),
      const SizedBox(height: 20),
      TextField(
        controller: _searchController,
        onChanged: (String value) => setState(() => _searchQuery = value),
        textInputAction: TextInputAction.search,
        decoration: InputDecoration(
          hintText: 'Cari judul, penulis, atau kategori',
          prefixIcon: const Icon(
            Icons.search_rounded,
            size: 20,
            color: AppTheme.textSecondary,
          ),
          suffixIcon: _searchQuery.isEmpty
              ? null
              : IconButton(
                  onPressed: () {
                    _searchController.clear();
                    setState(() => _searchQuery = '');
                  },
                  icon: const Icon(Icons.close_rounded, size: 18),
                  tooltip: 'Hapus pencarian',
                ),
        ),
      ),
      const SizedBox(height: 14),
      SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        child: Row(
          children: [
            for (final String category in _categories)
              Padding(
                padding: const EdgeInsets.only(right: 8),
                child: ChoiceChip(
                  label: Text(category),
                  selected: _selectedCategory == category,
```

```
                showCheckmark: false,
                onSelected: (_) {
                  setState(() => _selectedCategory = category);
                },
                selectedColor: AppTheme.primary,
                backgroundColor: Colors.white,
                labelStyle: TextStyle(
                  fontSize: 12,
                  fontWeight: FontWeight.w600,
                  color: _selectedCategory == category
                      ? Colors.white
                      : AppTheme.textSecondary,
                ),
                side: BorderSide(
                  color: _selectedCategory == category
                      ? AppTheme.primary
                      : AppTheme.border,
                ),
                shape: const StadiumBorder(),
              ),
            ),
          ],
        ),
      ),
      const SizedBox(height: 22),
      Row(
        children: [
          const Expanded(
            child: Text(
              'Koleksi Dokumen',
              style: TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.w700,
                color: AppTheme.textPrimary,
              ),
            ),
          ),
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
            decoration: BoxDecoration(
              color: AppTheme.primary.withValues(alpha: 0.08),
              borderRadius: BorderRadius.circular(20),
            ),
            child: Text(
              '${filteredDocuments.length} dokumen',
              style: const TextStyle(
                fontSize: 10.5,
                fontWeight: FontWeight.w600,
                color: AppTheme.primary,
              ),
            ),
          ),
        ],
      ),
      const SizedBox(height: 12),
      if (_errorMessage != null)
        _MessageCard(
          icon: Icons.error_outline_rounded,
          title: 'Terjadi kesalahan',
          message: _errorMessage!,
          actionLabel: 'Coba Lagi',
          onAction: _loadDocuments,
        )
      else if (filteredDocuments.isEmpty)
        _MessageCard(
          icon: Icons.search_off_rounded,
          title: 'Dokumen tidak ditemukan',
          message: _searchQuery.isEmpty && _selectedCategory == _allCategory
              ? 'Belum ada dokumen pada repositori.'
              : 'Coba kata kunci atau kategori lain.',
          actionLabel:
              _searchQuery.isEmpty && _selectedCategory == _allCategory
              ? null
              : 'Reset Filter',
          onAction: _searchQuery.isEmpty && _selectedCategory == _allCategory
              ? null
              : () {
```

```
                    _searchController.clear();
                    setState(() {
                      _searchQuery = '';
                      _selectedCategory = _allCategory;
                    });
                  },
            ),
          SizedBox(height: filteredDocuments.isEmpty ? 0 : 4),
        ],
      );
    }
  }

class _WelcomeCard extends StatelessWidget {
  const _WelcomeCard({
    required this.user,
    required this.totalDocuments,
    required this.totalFavorites,
  });

  final User user;
  final int totalDocuments;
  final int totalFavorites;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        gradient: const LinearGradient(
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
          colors: [AppTheme.primary, AppTheme.primaryDark],
        ),
        borderRadius: BorderRadius.circular(24),
        boxShadow: const [
          BoxShadow(
            color: Color(0x331E3A8A),
            blurRadius: 24,
            offset: Offset(0, 12),
          ),
        ],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      'Selamat datang kembali,',
                      style: TextStyle(
                        color: Colors.white.withValues(alpha: 0.72),
                        fontSize: 12,
                      ),
                    ),
                    const SizedBox(height: 4),
                    Text(
                      user.nama,
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                      style: const TextStyle(
                        color: Colors.white,
                        fontSize: 20,
                        fontWeight: FontWeight.w700,
                        height: 1.25,
                      ),
                    ),
                    const SizedBox(height: 10),
                    Container(
                      padding: const EdgeInsets.symmetric(
                        horizontal: 10,
                        vertical: 5,
```

```
                    ),
                    decoration: BoxDecoration(
                      color: Colors.white.withValues(alpha: 0.16),
                      borderRadius: BorderRadius.circular(20),
                    ),
                    child: Text(
                      '${AppConstants.studentNim} • ${AppConstants.studentClass}',
                      style: const TextStyle(
                        color: Colors.white,
                        fontSize: 10.5,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                ),
              ],
            ),
          ),
          const SizedBox(width: 12),
          Container(
            width: 54,
            height: 54,
            padding: const EdgeInsets.all(3),
            decoration: BoxDecoration(
              shape: BoxShape.circle,
              color: Colors.white.withValues(alpha: 0.2),
            ),
            child: ClipOval(
              child: Image.asset(
                AppConstants.profileImage,
                fit: BoxFit.cover,
              ),
            ),
          ),
        ],
      ),
      const SizedBox(height: 18),
      Divider(color: Colors.white.withValues(alpha: 0.18), height: 1),
      const SizedBox(height: 14),
      Row(
        children: [
          Expanded(
            child: _StatItem(
              label: 'Total Dokumen',
              value: '$totalDocuments',
              icon: Icons.description_outlined,
            ),
          ),
          Container(
            width: 1,
            height: 32,
            color: Colors.white.withValues(alpha: 0.18),
          ),
          Expanded(
            child: _StatItem(
              label: 'Favorit',
              value: '$totalFavorites',
              icon: Icons.favorite_border_rounded,
            ),
          ),
        ],
      ),
    ],
  ),
),
);
}
}

class _StatItem extends StatelessWidget {
const _StatItem({
required this.label,
required this.value,
required this.icon,
});

final String label;
final String value;
final IconData icon;
```

```
  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Icon(icon, color: Colors.white70, size: 20),
        const SizedBox(width: 10),
        Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              value,
              style: const TextStyle(
                color: Colors.white,
                fontSize: 16,
                fontWeight: FontWeight.w700,
              ),
            ),
            Text(
              label,
              style: TextStyle(
                color: Colors.white.withValues(alpha: 0.7),
                fontSize: 10.5,
              ),
            ),
          ],
        ),
      ],
    );
  }
}

class _MessageCard extends StatelessWidget {
  const _MessageCard({
    required this.icon,
    required this.title,
    required this.message,
    this.actionLabel,
    this.onAction,
  });

  final IconData icon;
  final String title;
  final String message;
  final String? actionLabel;
  final VoidCallback? onAction;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.symmetric(vertical: 28, horizontal: 24),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(18),
        border: Border.all(color: AppTheme.border),
      ),
      child: Column(
        children: [
          Icon(icon, size: 36, color: AppTheme.textSecondary),
          const SizedBox(height: 10),
          Text(
            title,
            textAlign: TextAlign.center,
            style: const TextStyle(
              fontWeight: FontWeight.w600,
              color: AppTheme.textPrimary,
            ),
          ),
          const SizedBox(height: 4),
          Text(
            message,
            textAlign: TextAlign.center,
            style: const TextStyle(
              fontSize: 12,
              color: AppTheme.textSecondary,
              height: 1.4,
```

```
            ),
          ),
          if (actionLabel != null && onAction != null) ...[
            const SizedBox(height: 12),
            TextButton(onPressed: onAction, child: Text(actionLabel!)),
          ],
        ],
      ),
    );
  }
}
```

lib/features/documents/document_detail_page.dart (440 baris)

```
import 'package:flutter/material.dart';

import '../../core/theme/app_theme.dart';
import '../../core/utils/date_formatter.dart';
import '../../core/widgets/app_feedback.dart';
import '../../models/document.dart';
import '../../repositories/document_repository.dart';

/// Halaman detail dokumen.
///
/// Menampilkan gambar sampul, judul, deskripsi, serta informasi tambahan
/// (kategori, tahun, penulis, dan tanggal ditambahkan). Tombol aksi favorit
/// memakai AlertDialog konfirmasi sebelum data diperbarui.
class DocumentDetailPage extends StatefulWidget {
  const DocumentDetailPage({super.key, required this.document});

  final Document document;

  @override
  State<DocumentDetailPage> createState() => _DocumentDetailPageState();
}

class _DocumentDetailPageState extends State<DocumentDetailPage> {
  late Document _document;
  bool _isUpdating = false;

  @override
  void initState() {
    super.initState();
    _document = widget.document;
  }

  Future<void> _toggleFavorite() async {
    final bool willAdd = !_document.isFavorite;

    final bool confirmed = await showConfirmDialog(
      context,
      title: willAdd ? 'Tambah ke Favorit' : 'Hapus dari Favorit',
      message: willAdd
          ? 'Yakin ingin menambahkan "${_document.title}" ke favorit?'
          : 'Yakin ingin menghapus "${_document.title}" dari favorit?',
    );

    if (!confirmed || !mounted) return;

    setState(() => _isUpdating = true);

    try {
      final int? id = _document.id;
      if (id == null) {
        throw StateError('Dokumen belum tersimpan pada database.');
      }

      await DocumentRepository.instance.setFavorite(id, willAdd);

      if (!mounted) return;
      setState(() {
        _document = _document.copyWith(isFavorite: willAdd);
        _isUpdating = false;
      });
```

```
    showAppSnackBar(
      context,
      willAdd
          ? 'Dokumen berhasil ditambahkan ke favorit'
          : 'Dokumen dihapus dari daftar favorit',
    );
  } catch (error) {
    if (!mounted) return;
    setState(() => _isUpdating = false);
    showAppDialog(
      context,
      title: 'Gagal Memperbarui',
      message: 'Terjadi kesalahan saat menyimpan perubahan: $error',
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: CustomScrollView(
      slivers: [
        SliverAppBar(
          pinned: true,
          expandedHeight: 300,
          backgroundColor: AppTheme.primaryDark,
          foregroundColor: Colors.white,
          flexibleSpace: FlexibleSpaceBar(
            background: _CoverHeader(document: _document),
          ),
        ),
        SliverToBoxAdapter(
          child: Padding(
            padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  _document.title,
                  style: const TextStyle(
                    fontSize: 21,
                    fontWeight: FontWeight.w700,
                    color: AppTheme.textPrimary,
                    height: 1.3,
                  ),
                ),
                const SizedBox(height: 14),
                Row(
                  children: [
                    Expanded(
                      child: _InfoChip(
                        icon: Icons.category_outlined,
                        label: _document.category,
                      ),
                    ),
                    const SizedBox(width: 10),
                    Expanded(
                      child: _InfoChip(
                        icon: Icons.calendar_today_outlined,
                        label: 'Tahun ${_document.year}',
                      ),
                    ),
                    if (_document.isFavorite) ...[
                      const SizedBox(width: 10),
                      const Expanded(
                        child: _InfoChip(
                          icon: Icons.favorite_rounded,
                          label: 'Favorit',
                          isHighlighted: true,
                        ),
                      ),
                    ],
                  ],
                ),
                const SizedBox(height: 20),
                Container(
```

```
                padding: const EdgeInsets.all(14),
                decoration: BoxDecoration(
                  color: Colors.white,
                  borderRadius: BorderRadius.circular(16),
                  border: Border.all(color: AppTheme.border),
                ),
                child: Row(
                  children: [
                    CircleAvatar(
                      radius: 20,
                      backgroundColor: AppTheme.primary.withValues(
                        alpha: 0.1,
                      ),
                      child: const Icon(
                        Icons.person_outline_rounded,
                        size: 20,
                        color: AppTheme.primary,
                      ),
                    ),
                    const SizedBox(width: 12),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          const Text(
                            'Penulis',
                            style: TextStyle(
                              fontSize: 11,
                              color: AppTheme.textSecondary,
                            ),
                          ),
                          Text(
                            _document.author,
                            style: const TextStyle(
                              fontSize: 13.5,
                              fontWeight: FontWeight.w600,
                              color: AppTheme.textPrimary,
                            ),
                          ),
                        ],
                      ),
                    ),
                    Text(
                      formatTanggalPanjang(_document.createdAt),
                      style: const TextStyle(
                        fontSize: 10.5,
                        color: AppTheme.textSecondary,
                      ),
                    ),
                  ],
                ),
              ),
            ),
            const SizedBox(height: 24),
            const Text(
              'Deskripsi',
              style: TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.w700,
                color: AppTheme.textPrimary,
              ),
            ),
            const SizedBox(height: 10),
            Text(
              _document.description,
              style: const TextStyle(
                fontSize: 13.5,
                height: 1.7,
                color: Color(0xFF334155),
              ),
            ),
            const SizedBox(height: 24),
            const Text(
              'Informasi Dokumen',
              style: TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.w700,
                color: AppTheme.textPrimary,
```

```
                    ),
                  ),
                  const SizedBox(height: 10),
                  Container(
                    decoration: BoxDecoration(
                      color: Colors.white,
                      borderRadius: BorderRadius.circular(16),
                      border: Border.all(color: AppTheme.border),
                    ),
                    child: Column(
                      children: [
                        _DetailRow(
                          label: 'Kategori',
                          value: _document.category,
                        ),
                        const Divider(height: 1, indent: 16, endIndent: 16),
                        _DetailRow(
                          label: 'Tahun Terbit',
                          value: '${_document.year}',
                        ),
                        const Divider(height: 1, indent: 16, endIndent: 16),
                        _DetailRow(label: 'Penulis', value: _document.author),
                        const Divider(height: 1, indent: 16, endIndent: 16),
                        _DetailRow(
                          label: 'Status',
                          value: _document.isFavorite
                              ? 'Favorit'
                              : 'Koleksi Umum',
                        ),
                      ],
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      ],
    ),
    bottomNavigationBar: SafeArea(
      top: false,
      child: Container(
        padding: const EdgeInsets.fromLTRB(20, 12, 20, 12),
        decoration: const BoxDecoration(
          color: Colors.white,
          border: Border(top: BorderSide(color: AppTheme.border)),
        ),
        child: _FavoriteButton(
          isFavorite: _document.isFavorite,
          isLoading: _isUpdating,
          onPressed: _toggleFavorite,
        ),
      ),
    ),
  );
 }
}

/// Bagian atas halaman detail berisi sampul dokumen dengan latar gradasi.
class _CoverHeader extends StatelessWidget {
  const _CoverHeader({required this.document});

  final Document document;

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
          colors: [AppTheme.primary, AppTheme.primaryDark],
        ),
      ),
      child: SafeArea(
        bottom: false,
        child: Center(
          child: Padding(
```

```
            padding: const EdgeInsets.only(top: 44, bottom: 18),
            child: Container(
              height: 200,
              width: 150,
              decoration: BoxDecoration(
                borderRadius: BorderRadius.circular(14),
                boxShadow: const [
                  BoxShadow(
                    color: Color(0x66000000),
                    blurRadius: 24,
                    offset: Offset(0, 12),
                  ),
                ],
                image: DecorationImage(
                  image: AssetImage(document.image),
                  fit: BoxFit.cover,
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

class _FavoriteButton extends StatelessWidget {
  const _FavoriteButton({
    required this.isFavorite,
    required this.isLoading,
    required this.onPressed,
  });

  final bool isFavorite;
  final bool isLoading;
  final VoidCallback onPressed;

  @override
  Widget build(BuildContext context) {
    return FilledButton.icon(
      onPressed: isLoading ? null : onPressed,
      icon: isLoading
          ? const SizedBox(
              width: 18,
              height: 18,
              child: CircularProgressIndicator(
                strokeWidth: 2,
                valueColor: AlwaysStoppedAnimation<Color>(Colors.white),
              ),
            )
          : Icon(
              isFavorite
                  ? Icons.favorite_rounded
                  : Icons.favorite_border_rounded,
              size: 20,
            ),
      label: Text(isFavorite ? 'Hapus dari Favorit' : 'Tambah ke Favorit'),
      style: FilledButton.styleFrom(
        backgroundColor: isFavorite
            ? const Color(0xFFE11D48)
            : AppTheme.primary,
      ),
    );
  }
}

class _InfoChip extends StatelessWidget {
  const _InfoChip({
    required this.icon,
    required this.label,
    this.isHighlighted = false,
  });

  final IconData icon;
  final String label;
  final bool isHighlighted;
```

```
  @override
  Widget build(BuildContext context) {
    final Color color = isHighlighted
        ? const Color(0xFFE11D48)
        : AppTheme.primary;

    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 10),
      decoration: BoxDecoration(
        color: color.withValues(alpha: 0.08),
        borderRadius: BorderRadius.circular(12),
      ),
      child: Row(
        children: [
          Icon(icon, size: 15, color: color),
          const SizedBox(width: 6),
          Expanded(
            child: Text(
              label,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: TextStyle(
                fontSize: 11,
                fontWeight: FontWeight.w600,
                color: color,
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class _DetailRow extends StatelessWidget {
  const _DetailRow({required this.label, required this.value});

  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 13),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          SizedBox(
            width: 110,
            child: Text(
              label,
              style: const TextStyle(
                fontSize: 12,
                color: AppTheme.textSecondary,
              ),
            ),
          ),
          Expanded(
            child: Text(
              value,
              style: const TextStyle(
                fontSize: 12.5,
                fontWeight: FontWeight.w600,
                color: AppTheme.textPrimary,
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

lib/features/documents/document_form_page.dart (323 baris)

```
import 'package:flutter/material.dart';

import '../../core/constants/app_constants.dart';
import '../../core/theme/app_theme.dart';
import '../../core/utils/validators.dart';
import '../../core/widgets/app_feedback.dart';
import '../../core/widgets/app_text_field.dart';
import '../../core/widgets/primary_button.dart';
import '../../models/document.dart';
import '../../repositories/document_repository.dart';

/// Halaman form untuk menambah dan mengubah dokumen.
///
/// Jika [document] bernilai `null`, halaman berjalan dalam mode tambah data.
class DocumentFormPage extends StatefulWidget {
  const DocumentFormPage({super.key, this.document});

  final Document? document;

  bool get isEditing => document != null;

  @override
  State<DocumentFormPage> createState() => _DocumentFormPageState();
}

class _DocumentFormPageState extends State<DocumentFormPage> {
  static const List<String> _categories = [
    'Teknologi',
    'Akademik',
    'Manajemen',
    'Sains',
    'Bisnis',
    'Umum',
  ];

  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
  final TextEditingController _titleController = TextEditingController();
  final TextEditingController _authorController = TextEditingController();
  final TextEditingController _yearController = TextEditingController();
  final TextEditingController _descriptionController = TextEditingController();

  late String _category;
  late String _selectedImage;
  bool _isSaving = false;

  @override
  void initState() {
    super.initState();
    final Document? document = widget.document;

    _titleController.text = document?.title ?? '';
    _authorController.text = document?.author ?? '';
    _yearController.text = document?.year.toString() ?? '';
    _descriptionController.text = document?.description ?? '';
    _category = document?.category ?? _categories.first;
    _selectedImage = document?.image ?? AppConstants.coverImages.first;
  }

  @override
  void dispose() {
    _titleController.dispose();
    _authorController.dispose();
    _yearController.dispose();
    _descriptionController.dispose();
    super.dispose();
  }

  Future<void> _saveDocument() async {
    FocusScope.of(context).unfocus();

    if (!(_formKey.currentState?.validate() ?? false)) return;

    setState(() => _isSaving = true);
```

```
    final DateTime now = DateTime.now();
    final Document document = Document(
      id: widget.document?.id,
      title: _titleController.text.trim(),
      category: _category,
      author: _authorController.text.trim(),
      year: int.parse(_yearController.text.trim()),
      description: _descriptionController.text.trim(),
      image: _selectedImage,
      isFavorite: widget.document?.isFavorite ?? false,
      createdAt: widget.document?.createdAt ?? now,
      updatedAt: now,
    );

    try {
      if (widget.isEditing) {
        await DocumentRepository.instance.update(document);
      } else {
        await DocumentRepository.instance.create(document);
      }

      if (!mounted) return;
      showAppSnackBar(
        context,
        widget.isEditing
            ? 'Dokumen berhasil diperbarui'
            : 'Dokumen berhasil ditambahkan',
      );
      Navigator.of(context).pop(true);
    } catch (error) {
      if (!mounted) return;
      setState(() => _isSaving = false);
      showAppDialog(
        context,
        title: 'Gagal Menyimpan',
        message: 'Terjadi kesalahan saat menyimpan dokumen: $error',
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    final List<String> categories = <String>{
      ..._categories,
      if (widget.document != null) widget.document!.category,
    }.toList();

    return Scaffold(
      appBar: AppBar(
        title: Text(widget.isEditing ? 'Edit Dokumen' : 'Tambah Dokumen'),
      ),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
          children: [
            Container(
              padding: const EdgeInsets.all(18),
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(20),
                border: Border.all(color: AppTheme.border),
              ),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  AppTextField(
                    label: 'Judul Dokumen',
                    hint: 'Contoh: Basis Data Relasional',
                    controller: _titleController,
                    prefixIcon: Icons.title_rounded,
                    validator: (String? value) =>
                        AppValidators.requiredField(value, 'Judul'),
                    textInputAction: TextInputAction.next,
                  ),
                  const SizedBox(height: 16),
```

```
                AppTextField(
                  label: 'Penulis',
                  hint: 'Nama penulis dokumen',
                  controller: _authorController,
                  prefixIcon: Icons.person_outline_rounded,
                  validator: (String? value) =>
                      AppValidators.requiredField(value, 'Penulis'),
                  textInputAction: TextInputAction.next,
                ),
                const SizedBox(height: 16),
                Row(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          const Text(
                            'Kategori',
                            style: TextStyle(
                              fontSize: 13,
                              fontWeight: FontWeight.w600,
                              color: Color(0xFF334155),
                            ),
                          ),
                          const SizedBox(height: 8),
                          DropdownButtonFormField<String>(
                            initialValue: _category,
                            isExpanded: true,
                            decoration: const InputDecoration(
                              prefixIcon: Icon(
                                Icons.category_outlined,
                                size: 20,
                                color: Color(0xFF64748B),
                              ),
                            ),
                            items: [
                              for (final String category in categories)
                                DropdownMenuItem<String>(
                                  value: category,
                                  child: Text(
                                    category,
                                    style: const TextStyle(fontSize: 14),
                                  ),
                                ),
                            ],
                            onChanged: (String? value) {
                              if (value != null) {
                                setState(() => _category = value);
                              }
                            },
                          ),
                        ],
                      ),
                    ),
                    const SizedBox(width: 12),
                    SizedBox(
                      width: 110,
                      child: AppTextField(
                        label: 'Tahun',
                        hint: '2025',
                        controller: _yearController,
                        keyboardType: TextInputType.number,
                        validator: AppValidators.tahun,
                        textInputAction: TextInputAction.next,
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 16),
                AppTextField(
                  label: 'Deskripsi',
                  hint: 'Ringkasan isi dokumen',
                  controller: _descriptionController,
                  maxLines: 5,
                  validator: (String? value) =>
                      AppValidators.requiredField(value, 'Deskripsi'),
```

```
                  textInputAction: TextInputAction.newline,
                ),
              ],
            ),
          ),
        ),
        const SizedBox(height: 20),
        const Text(
          'Sampul Dokumen',
          style: TextStyle(
            fontSize: 14,
            fontWeight: FontWeight.w700,
            color: AppTheme.textPrimary,
          ),
        ),
        const SizedBox(height: 4),
        const Text(
          'Pilih salah satu gambar dari aset lokal aplikasi.',
          style: TextStyle(fontSize: 11.5, color: AppTheme.textSecondary),
        ),
        const SizedBox(height: 12),
        SizedBox(
          height: 142,
          child: ListView.separated(
            scrollDirection: Axis.horizontal,
            itemCount: AppConstants.coverImages.length,
            separatorBuilder: (_, _) => const SizedBox(width: 12),
            itemBuilder: (BuildContext context, int index) {
              final String image = AppConstants.coverImages[index];
              final bool isSelected = image == _selectedImage;

              return GestureDetector(
                onTap: () => setState(() => _selectedImage = image),
                child: Container(
                  width: 94,
                  decoration: BoxDecoration(
                    borderRadius: BorderRadius.circular(14),
                    border: Border.all(
                      color: isSelected
                          ? AppTheme.primary
                          : AppTheme.border,
                      width: isSelected ? 2.5 : 1,
                    ),
                  ),
                  child: ClipRRect(
                    borderRadius: BorderRadius.circular(12),
                    child: Stack(
                      fit: StackFit.expand,
                      children: [
                        Image.asset(image, fit: BoxFit.cover),
                        if (isSelected)
                          Positioned(
                            top: 6,
                            right: 6,
                            child: Container(
                              padding: const EdgeInsets.all(3),
                              decoration: const BoxDecoration(
                                color: AppTheme.primary,
                                shape: BoxShape.circle,
                              ),
                              child: const Icon(
                                Icons.check_rounded,
                                size: 14,
                                color: Colors.white,
                              ),
                            ),
                          ),
                      ],
                    ),
                  ),
                ),
              );
            },
          ),
        ),
      ],
    ),
  ),
```

```
      bottomNavigationBar: SafeArea(
        top: false,
        child: Container(
          padding: const EdgeInsets.fromLTRB(20, 12, 20, 12),
          decoration: const BoxDecoration(
            color: Colors.white,
            border: Border(top: BorderSide(color: AppTheme.border)),
          ),
          child: PrimaryButton(
            label: widget.isEditing ? 'Simpan Perubahan' : 'Tambah Dokumen',
            icon: Icons.save_outlined,
            isLoading: _isSaving,
            onPressed: _saveDocument,
          ),
        ),
      ),
    );
  }
}
```

lib/features/documents/widgets/document_card.dart (150 baris)

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_theme.dart';
import '../../../models/document.dart';

/// Kartu ringkas dokumen yang dipakai pada daftar dashboard, favorit,
/// dan halaman pengaturan.
class DocumentCard extends StatelessWidget {
  const DocumentCard({
    super.key,
    required this.document,
    this.onTap,
    this.trailing,
  });

  final Document document;
  final VoidCallback? onTap;
  final Widget? trailing;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.white,
      borderRadius: BorderRadius.circular(18),
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(18),
        child: Container(
          padding: const EdgeInsets.all(12),
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(18),
            border: Border.all(color: AppTheme.border),
          ),
          child: Row(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              ClipRRect(
                borderRadius: BorderRadius.circular(12),
                child: Image.asset(
                  document.image,
                  width: 64,
                  height: 88,
                  fit: BoxFit.cover,
                  errorBuilder: (_, _, _) => Container(
                    width: 64,
                    height: 88,
                    color: AppTheme.scaffold,
                    child: const Icon(
                      Icons.image_not_supported_outlined,
                      size: 20,
                      color: AppTheme.textSecondary,
                    ),
```

```
      ),
    ),
  ),
  const SizedBox(width: 14),
  Expanded(
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          children: [
            Expanded(
              child: Text(
                document.title,
                maxLines: 2,
                overflow: TextOverflow.ellipsis,
                style: const TextStyle(
                  fontSize: 14,
                  fontWeight: FontWeight.w700,
                  color: AppTheme.textPrimary,
                  height: 1.3,
                ),
              ),
            ),
            if (document.isFavorite)
              const Padding(
                padding: EdgeInsets.only(left: 6),
                child: Icon(
                  Icons.favorite_rounded,
                  size: 16,
                  color: Color(0xFFE11D48),
                ),
              ),
          ],
        ),
        const SizedBox(height: 6),
        Row(
          children: [
            Container(
              padding: const EdgeInsets.symmetric(
                horizontal: 8,
                vertical: 3,
              ),
              decoration: BoxDecoration(
                color: AppTheme.primary.withValues(alpha: 0.08),
                borderRadius: BorderRadius.circular(8),
              ),
              child: Text(
                document.category,
                style: const TextStyle(
                  fontSize: 10,
                  fontWeight: FontWeight.w600,
                  color: AppTheme.primary,
                ),
              ),
            ),
            const SizedBox(width: 8),
            Text(
              '${document.year}',
              style: const TextStyle(
                fontSize: 10.5,
                color: AppTheme.textSecondary,
              ),
            ),
          ],
        ),
        const SizedBox(height: 8),
        Row(
          children: [
            const Icon(
              Icons.person_outline_rounded,
              size: 14,
              color: AppTheme.textSecondary,
            ),
            const SizedBox(width: 4),
            Expanded(
              child: Text(
                document.author,
```

```
                          maxLines: 1,
                          overflow: TextOverflow.ellipsis,
                          style: const TextStyle(
                            fontSize: 11.5,
                            color: AppTheme.textSecondary,
                          ),
                        ),
                      ),
                      ?trailing,
                    ],
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/features/favorites/favorites_page.dart (211 baris)

```
import 'package:flutter/material.dart';

import '../../core/theme/app_theme.dart';
import '../../models/document.dart';
import '../../repositories/document_repository.dart';
import '../documents/document_detail_page.dart';
import '../documents/widgets/document_card.dart';

/// Halaman daftar dokumen favorit.
///
/// Data dibaca dari kolom `is_favorite` pada tabel dokumen sehingga daftar
/// selalu sinkron dengan aksi favorit pada halaman detail.
class FavoritesPage extends StatefulWidget {
  const FavoritesPage({super.key});

  @override
  State<FavoritesPage> createState() => _FavoritesPageState();
}

class _FavoritesPageState extends State<FavoritesPage> {
  final DocumentRepository _repository = DocumentRepository.instance;

  List<Document> _documents = [];
  bool _isLoading = true;
  String? _errorMessage;

  @override
  void initState() {
    super.initState();
    _loadFavorites();
  }

  Future<void> _loadFavorites() async {
    setState(() {
      _isLoading = true;
      _errorMessage = null;
    });

    try {
      final List<Document> documents = await _repository.getFavorites();
      if (!mounted) return;
      setState(() {
        _documents = documents;
        _isLoading = false;
      });
    } catch (error) {
      if (!mounted) return;
      setState(() {
        _errorMessage = 'Gagal memuat favorit: $error';
        _isLoading = false;
```

```
    });
  }
}

Future<void> _openDetail(Document document) async {
  await Navigator.of(context).push(
    MaterialPageRoute<void>(
      builder: (BuildContext context) =>
          DocumentDetailPage(document: document),
    ),
  );

  if (!mounted) return;
  await _loadFavorites();
}

@override
Widget build(BuildContext context) {
  return RefreshIndicator(
    onRefresh: _loadFavorites,
    color: AppTheme.primary,
    child: ListView.builder(
      padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
      physics: const AlwaysScrollableScrollPhysics(),
      itemCount: _documents.length + 1,
      itemBuilder: (BuildContext context, int index) {
        if (index == 0) return _buildHeader();

        final Document document = _documents[index - 1];
        return Padding(
          padding: const EdgeInsets.only(bottom: 12),
          child: DocumentCard(
            document: document,
            onTap: () => _openDetail(document),
          ),
        );
      },
    ),
  );
}

Widget _buildHeader() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Row(
        children: [
          const Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  'Favorit Saya',
                  style: TextStyle(
                    fontSize: 18,
                    fontWeight: FontWeight.w700,
                    color: AppTheme.textPrimary,
                  ),
                ),
                SizedBox(height: 4),
                Text(
                  'Dokumen yang Anda tandai sebagai favorit.',
                  style: TextStyle(
                    fontSize: 12.5,
                    color: AppTheme.textSecondary,
                  ),
                ),
              ],
            ),
          ),
          if (!_isLoading)
            Container(
              padding: const EdgeInsets.symmetric(
                horizontal: 10,
                vertical: 5,
              ),
              decoration: BoxDecoration(
```

```
                  color: const Color(0xFFE11D48).withValues(alpha: 0.08),
                  borderRadius: BorderRadius.circular(20),
                ),
                child: Text(
                  '${_documents.length} dokumen',
                  style: const TextStyle(
                    fontSize: 10.5,
                    fontWeight: FontWeight.w600,
                    color: Color(0xFFE11D48),
                  ),
                ),
              ),
            ),
          ],
        ),
        const SizedBox(height: 20),
        if (_isLoading)
          const Padding(
            padding: EdgeInsets.symmetric(vertical: 48),
            child: Center(child: CircularProgressIndicator()),
          )
        else if (_errorMessage != null)
          _FavoritesMessage(
            icon: Icons.error_outline_rounded,
            message: _errorMessage!,
            actionLabel: 'Coba Lagi',
            onAction: _loadFavorites,
          )
        else if (_documents.isEmpty)
          const _FavoritesMessage(
            icon: Icons.favorite_border_rounded,
            message:
                'Belum ada dokumen favorit. Buka detail dokumen lalu tekan '
                'tombol "Tambah ke Favorit" untuk menambahkannya ke sini.',
          ),
      ],
    );
  }
}

class _FavoritesMessage extends StatelessWidget {
  const _FavoritesMessage({
    required this.icon,
    required this.message,
    this.actionLabel,
    this.onAction,
  });

  final IconData icon;
  final String message;
  final String? actionLabel;
  final VoidCallback? onAction;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.symmetric(vertical: 44, horizontal: 24),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(20),
        border: Border.all(color: AppTheme.border),
      ),
      child: Column(
        children: [
          Icon(icon, size: 40, color: AppTheme.textSecondary),
          const SizedBox(height: 12),
          Text(
            message,
            textAlign: TextAlign.center,
            style: const TextStyle(
              fontSize: 12.5,
              height: 1.5,
              color: AppTheme.textSecondary,
            ),
          ),
          if (actionLabel != null && onAction != null) ...[
            const SizedBox(height: 12),
```

```
            TextButton(onPressed: onAction, child: Text(actionLabel!)),
          ],
        ],
      ),
    );
  }
}
```

lib/features/profile/profile_page.dart (210 baris)

```
import 'package:flutter/material.dart';

import '../../core/constants/app_constants.dart';
import '../../core/theme/app_theme.dart';
import '../../core/widgets/app_feedback.dart';
import '../../models/user.dart';
import '../auth/login_page.dart';

/// Halaman profil berisi identitas mahasiswa dan tombol keluar.
class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key, required this.user});

  final User user;

  Future<void> _logout(BuildContext context) async {
    final bool confirmed = await showConfirmDialog(
      context,
      title: 'Keluar',
      message: 'Yakin ingin keluar dari aplikasi Rizal Docs?',
    );

    if (!confirmed || !context.mounted) return;

    Navigator.of(context).pushAndRemoveUntil(
      MaterialPageRoute<void>(
        builder: (BuildContext context) => const LoginPage(),
      ),
      (Route<dynamic> route) => false,
    );
  }

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
      children: [
        Container(
          padding: const EdgeInsets.all(20),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(22),
            border: Border.all(color: AppTheme.border),
          ),
          child: Column(
            children: [
              Container(
                width: 96,
                height: 96,
                padding: const EdgeInsets.all(4),
                decoration: const BoxDecoration(
                  shape: BoxShape.circle,
                  color: AppTheme.scaffold,
                ),
                child: ClipOval(
                  child: Image.asset(
                    AppConstants.profileImage,
                    fit: BoxFit.cover,
                  ),
                ),
              ),
              const SizedBox(height: 14),
              Text(
                user.nama,
                textAlign: TextAlign.center,
```

```
            style: const TextStyle(
              fontSize: 18,
              fontWeight: FontWeight.w700,
              color: AppTheme.textPrimary,
            ),
          ),
          const SizedBox(height: 4),
          Text(
            '@${user.username}',
            style: const TextStyle(
              fontSize: 12.5,
              color: AppTheme.textSecondary,
            ),
          ),
        ],
      ),
    ),
    const SizedBox(height: 20),
    const Text(
      'Data Mahasiswa',
      style: TextStyle(
        fontSize: 16,
        fontWeight: FontWeight.w700,
        color: AppTheme.textPrimary,
      ),
    ),
    const SizedBox(height: 12),
    Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(20),
        border: Border.all(color: AppTheme.border),
      ),
      child: Column(
        children: [
          _InfoTile(
            icon: Icons.badge_outlined,
            label: 'Nama Lengkap',
            value: AppConstants.studentName,
          ),
          const Divider(height: 1, indent: 56),
          _InfoTile(
            icon: Icons.tag_rounded,
            label: 'NIM',
            value: AppConstants.studentNim,
          ),
          const Divider(height: 1, indent: 56),
          _InfoTile(
            icon: Icons.groups_outlined,
            label: 'Kelas',
            value: AppConstants.studentClass,
          ),
          const Divider(height: 1, indent: 56),
          _InfoTile(
            icon: Icons.school_outlined,
            label: 'Program Studi',
            value: AppConstants.studentProgram,
          ),
          const Divider(height: 1, indent: 56),
          _InfoTile(
            icon: Icons.location_city_rounded,
            label: 'Kampus',
            value: AppConstants.studentCampus,
          ),
        ],
      ),
    ),
    const SizedBox(height: 20),
    Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(20),
        border: Border.all(color: AppTheme.border),
      ),
      child: _InfoTile(
        icon: Icons.info_outline_rounded,
        label: 'Tentang Aplikasi',
```

```
            value:
                '${AppConstants.appName} v${AppConstants.appVersion} - '
                '${AppConstants.appTagline}',
          ),
        ),
        const SizedBox(height: 24),
        OutlinedButton.icon(
          onPressed: () => _logout(context),
          icon: const Icon(Icons.logout_rounded, size: 18),
          label: const Text('Keluar dari Akun'),
          style: OutlinedButton.styleFrom(
            foregroundColor: const Color(0xFFDC2626),
            side: const BorderSide(color: Color(0xFFFECACA)),
            backgroundColor: const Color(0xFFFEF2F2),
          ),
        ),
      ],
    );
  }
}

class _InfoTile extends StatelessWidget {
  const _InfoTile({
    required this.icon,
    required this.label,
    required this.value,
  });

  final IconData icon;
  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Icon(icon, size: 20, color: AppTheme.primary),
          const SizedBox(width: 14),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  label,
                  style: const TextStyle(
                    fontSize: 11,
                    color: AppTheme.textSecondary,
                  ),
                ),
                const SizedBox(height: 3),
                Text(
                  value,
                  style: const TextStyle(
                    fontSize: 13.5,
                    fontWeight: FontWeight.w600,
                    color: AppTheme.textPrimary,
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

lib/features/settings/settings_page.dart (492 baris)

```
import 'package:flutter/material.dart';
```

```dart
import '../../core/theme/app_theme.dart';
import '../../core/widgets/app_feedback.dart';
import '../../models/document.dart';
import '../../repositories/document_repository.dart';
import '../documents/document_form_page.dart';

/// Halaman pengaturan untuk mengelola data dokumen (create, read, update,
/// delete).
class SettingsPage extends StatefulWidget {
  const SettingsPage({super.key});

  @override
  State<SettingsPage> createState() => _SettingsPageState();
}

class _SettingsPageState extends State<SettingsPage> {
  final DocumentRepository _repository = DocumentRepository.instance;

  List<Document> _documents = [];
  bool _isLoading = true;
  String? _errorMessage;

  @override
  void initState() {
    super.initState();
    _loadDocuments();
  }

  Future<void> _loadDocuments() async {
    setState(() {
      _isLoading = true;
      _errorMessage = null;
    });

    try {
      final List<Document> documents = await _repository.getAll();
      if (!mounted) return;
      setState(() {
        _documents = documents;
        _isLoading = false;
      });
    } catch (error) {
      if (!mounted) return;
      setState(() {
        _errorMessage = 'Gagal memuat dokumen: $error';
        _isLoading = false;
      });
    }
  }

  Future<void> _openForm({Document? document}) async {
    final bool? changed = await Navigator.of(context).push<bool>(
      MaterialPageRoute<bool>(
        builder: (BuildContext context) => DocumentFormPage(document: document),
      ),
    );

    if (changed == true) {
      await _loadDocuments();
    }
  }

  Future<void> _deleteDocument(Document document) async {
    final bool confirmed = await showConfirmDialog(
      context,
      title: 'Hapus Dokumen',
      message:
          'Yakin ingin menghapus "${document.title}"? '
          'Data yang dihapus tidak dapat dikembalikan.',
    );

    if (!confirmed || !mounted) return;

    try {
      final int? id = document.id;
      if (id == null) {
        throw StateError('Dokumen tidak memiliki id.');
```

```
      }

      await _repository.delete(id);
      if (!mounted) return;
      showAppSnackBar(context, 'Dokumen berhasil dihapus');
      await _loadDocuments();
    } catch (error) {
      if (!mounted) return;
      showAppDialog(
        context,
        title: 'Gagal Menghapus',
        message: 'Terjadi kesalahan saat menghapus dokumen: $error',
      );
    }
  }

  void _showManageSheet() {
    showModalBottomSheet<void>(
      context: context,
      backgroundColor: Colors.white,
      shape: const RoundedRectangleBorder(
        borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
      ),
      builder: (BuildContext sheetContext) {
        return SafeArea(
          child: Padding(
            padding: const EdgeInsets.fromLTRB(20, 12, 20, 20),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Center(
                  child: Container(
                    width: 44,
                    height: 4,
                    decoration: BoxDecoration(
                      color: AppTheme.border,
                      borderRadius: BorderRadius.circular(4),
                    ),
                  ),
                ),
                const SizedBox(height: 18),
                const Text(
                  'Kelola Data Dokumen',
                  style: TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w700,
                    color: AppTheme.textPrimary,
                  ),
                ),
                const SizedBox(height: 6),
                const Text(
                  'Pilih tindakan yang ingin dilakukan pada koleksi dokumen.',
                  style: TextStyle(fontSize: 12, color: AppTheme.textSecondary),
                ),
                const SizedBox(height: 18),
                ListTile(
                  contentPadding: EdgeInsets.zero,
                  leading: const _SheetIcon(
                    icon: Icons.add_rounded,
                    color: AppTheme.primary,
                  ),
                  title: const Text('Tambah Dokumen'),
                  subtitle: const Text(
                    'Create - menambahkan data baru',
                    style: TextStyle(fontSize: 11),
                  ),
                  onTap: () {
                    Navigator.of(sheetContext).pop();
                    _openForm();
                  },
                ),
                ListTile(
                  contentPadding: EdgeInsets.zero,
                  leading: const _SheetIcon(
                    icon: Icons.edit_outlined,
                    color: Color(0xFF0F766E),
```

```
                ),
                title: const Text('Ubah Dokumen'),
                subtitle: const Text(
                  'Update - pilih dokumen pada daftar di bawah',
                  style: TextStyle(fontSize: 11),
                ),
                onTap: () => Navigator.of(sheetContext).pop(),
              ),
              ListTile(
                contentPadding: EdgeInsets.zero,
                leading: const _SheetIcon(
                  icon: Icons.delete_outline_rounded,
                  color: Color(0xFFDC2626),
                ),
                title: const Text('Hapus Dokumen'),
                subtitle: const Text(
                  'Delete - pilih ikon hapus pada daftar dokumen',
                  style: TextStyle(fontSize: 11),
                ),
                onTap: () => Navigator.of(sheetContext).pop(),
              ),
            ],
          ),
        ),
      );
    },
  );
}

@override
Widget build(BuildContext context) {
  return RefreshIndicator(
    onRefresh: _loadDocuments,
    color: AppTheme.primary,
    child: ListView(
      padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
      physics: const AlwaysScrollableScrollPhysics(),
      children: [
        Row(
          children: [
            const Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    'Pengaturan',
                    style: TextStyle(
                      fontSize: 18,
                      fontWeight: FontWeight.w700,
                      color: AppTheme.textPrimary,
                    ),
                  ),
                  SizedBox(height: 4),
                  Text(
                    'Kelola data koleksi dokumen aplikasi.',
                    style: TextStyle(
                      fontSize: 12.5,
                      color: AppTheme.textSecondary,
                    ),
                  ),
                ],
              ),
            ),
            IconButton(
              onPressed: _showManageSheet,
              icon: const Icon(Icons.more_horiz_rounded),
              tooltip: 'Menu pengelolaan data',
              style: IconButton.styleFrom(
                backgroundColor: Colors.white,
                side: const BorderSide(color: AppTheme.border),
              ),
            ),
          ],
        ),
        const SizedBox(height: 18),
        FilledButton.icon(
          onPressed: () => _openForm(),
```

```
            icon: const Icon(Icons.add_rounded, size: 20),
            label: const Text('Tambah Dokumen'),
          ),
          const SizedBox(height: 24),
          Row(
            children: [
              const Expanded(
                child: Text(
                  'Daftar Dokumen',
                  style: TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w700,
                    color: AppTheme.textPrimary,
                  ),
                ),
              ),
              Container(
                padding: const EdgeInsets.symmetric(
                  horizontal: 10,
                  vertical: 5,
                ),
                decoration: BoxDecoration(
                  color: AppTheme.primary.withValues(alpha: 0.08),
                  borderRadius: BorderRadius.circular(20),
                ),
                child: Text(
                  '${_documents.length} data',
                  style: const TextStyle(
                    fontSize: 10.5,
                    fontWeight: FontWeight.w600,
                    color: AppTheme.primary,
                  ),
                ),
              ),
            ],
          ),
          const SizedBox(height: 12),
          if (_isLoading)
            const Padding(
              padding: EdgeInsets.symmetric(vertical: 48),
              child: Center(child: CircularProgressIndicator()),
            )
          else if (_errorMessage != null)
            _SettingsMessage(
              icon: Icons.error_outline_rounded,
              message: _errorMessage!,
              actionLabel: 'Coba Lagi',
              onAction: _loadDocuments,
            )
          else if (_documents.isEmpty)
            const _SettingsMessage(
              icon: Icons.folder_off_outlined,
              message:
                  'Belum ada dokumen. Gunakan tombol Tambah Dokumen untuk '
                  'membuat data baru.',
            )
          else
            ..._documents.map(
              (Document document) => Padding(
                padding: const EdgeInsets.only(bottom: 10),
                child: _ManagedDocumentTile(
                  document: document,
                  onEdit: () => _openForm(document: document),
                  onDelete: () => _deleteDocument(document),
                ),
              ),
            ),
          const SizedBox(height: 12),
          Container(
            padding: const EdgeInsets.all(16),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(18),
              border: Border.all(color: AppTheme.border),
            ),
            child: const Row(
              children: [
```

```
                Icon(
                  Icons.info_outline_rounded,
                  size: 18,
                  color: AppTheme.primary,
                ),
                SizedBox(width: 10),
                Expanded(
                  child: Text(
                    'Menu pengelolaan data mencakup tambah (create), lihat '
                    '(read), ubah (update), dan hapus (delete) dokumen.',
                    style: TextStyle(
                      fontSize: 11.5,
                      color: AppTheme.textSecondary,
                      height: 1.45,
                    ),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

class _ManagedDocumentTile extends StatelessWidget {
  const _ManagedDocumentTile({
    required this.document,
    required this.onEdit,
    required this.onDelete,
  });

  final Document document;
  final VoidCallback onEdit;
  final VoidCallback onDelete;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.white,
      clipBehavior: Clip.antiAlias,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(18),
        side: const BorderSide(color: AppTheme.border),
      ),
      child: ListTile(
        contentPadding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
        onTap: onEdit,
        leading: ClipRRect(
          borderRadius: BorderRadius.circular(10),
          child: Image.asset(
            document.image,
            width: 44,
            height: 58,
            fit: BoxFit.cover,
          ),
        ),
        title: Text(
          document.title,
          maxLines: 2,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(
            fontSize: 13,
            fontWeight: FontWeight.w600,
            color: AppTheme.textPrimary,
          ),
        ),
        subtitle: Padding(
          padding: const EdgeInsets.only(top: 4),
          child: Text(
            '${document.category} • ${document.year}',
            style: const TextStyle(fontSize: 11, color: AppTheme.textSecondary),
          ),
        ),
        trailing: PopupMenuButton<String>(
          icon: const Icon(Icons.more_vert_rounded, size: 20),
```

```
          tooltip: 'Aksi dokumen',
          onSelected: (String value) {
            if (value == 'edit') {
              onEdit();
            } else if (value == 'delete') {
              onDelete();
            }
          },
          itemBuilder: (BuildContext context) => const [
            PopupMenuItem<String>(
              value: 'edit',
              child: Row(
                children: [
                  Icon(Icons.edit_outlined, size: 18),
                  SizedBox(width: 10),
                  Text('Ubah'),
                ],
              ),
            ),
            PopupMenuItem<String>(
              value: 'delete',
              child: Row(
                children: [
                  Icon(
                    Icons.delete_outline_rounded,
                    size: 18,
                    color: Color(0xFFDC2626),
                  ),
                  SizedBox(width: 10),
                  Text('Hapus', style: TextStyle(color: Color(0xFFDC2626))),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class _SettingsMessage extends StatelessWidget {
  const _SettingsMessage({
    required this.icon,
    required this.message,
    this.actionLabel,
    this.onAction,
  });

  final IconData icon;
  final String message;
  final String? actionLabel;
  final VoidCallback? onAction;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.symmetric(vertical: 36, horizontal: 24),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(18),
        border: Border.all(color: AppTheme.border),
      ),
      child: Column(
        children: [
          Icon(icon, size: 38, color: AppTheme.textSecondary),
          const SizedBox(height: 12),
          Text(
            message,
            textAlign: TextAlign.center,
            style: const TextStyle(
              fontSize: 12.5,
              color: AppTheme.textSecondary,
              height: 1.45,
            ),
          ),
          if (actionLabel != null && onAction != null) ...[
```

```
            const SizedBox(height: 10),
            TextButton(onPressed: onAction, child: Text(actionLabel!)),
          ],
        ],
      ),
    );
  }
}

class _SheetIcon extends StatelessWidget {
  const _SheetIcon({required this.icon, required this.color});

  final IconData icon;
  final Color color;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 40,
      height: 40,
      decoration: BoxDecoration(
        color: color.withValues(alpha: 0.1),
        borderRadius: BorderRadius.circular(12),
      ),
      child: Icon(icon, color: color, size: 20),
    );
  }
}
```

lib/features/splash/splash_page.dart (128 baris)

```
import 'dart:async';

import 'package:flutter/material.dart';

import '../../core/constants/app_constants.dart';
import '../../core/theme/app_theme.dart';
import '../auth/login_page.dart';

/// Halaman pembuka aplikasi.
///
/// Menampilkan branding singkat sebelum pengguna diarahkan ke halaman login.
class SplashPage extends StatefulWidget {
  const SplashPage({super.key});

  @override
  State<SplashPage> createState() => _SplashPageState();
}

class _SplashPageState extends State<SplashPage> {
  Timer? _timer;

  @override
  void initState() {
    super.initState();
    _timer = Timer(const Duration(milliseconds: 1600), _openLoginPage);
  }

  void _openLoginPage() {
    if (!mounted) return;
    Navigator.of(context).pushReplacement(
      MaterialPageRoute<void>(
        builder: (BuildContext context) => const LoginPage(),
      ),
    );
  }

  @override
  void dispose() {
    _timer?.cancel();
    super.dispose();
  }

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        width: double.infinity,
        decoration: const BoxDecoration(
          gradient: LinearGradient(
            begin: Alignment.topCenter,
            end: Alignment.bottomCenter,
            colors: [AppTheme.primaryDark, AppTheme.primary],
          ),
        ),
        child: SafeArea(
          child: Column(
            children: [
              const Spacer(),
              Container(
                width: 112,
                height: 112,
                padding: const EdgeInsets.all(14),
                decoration: BoxDecoration(
                  color: Colors.white.withValues(alpha: 0.14),
                  shape: BoxShape.circle,
                ),
                child: Image.asset(AppConstants.logoImage),
              ),
              const SizedBox(height: 24),
              const Text(
                AppConstants.appName,
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 28,
                  fontWeight: FontWeight.w700,
                  letterSpacing: 0.4,
                ),
              ),
              const SizedBox(height: 8),
              Text(
                AppConstants.appTagline,
                style: TextStyle(
                  color: Colors.white.withValues(alpha: 0.78),
                  fontSize: 14,
                ),
              ),
              const SizedBox(height: 36),
              SizedBox(
                width: 28,
                height: 28,
                child: CircularProgressIndicator(
                  strokeWidth: 2.6,
                  valueColor: AlwaysStoppedAnimation<Color>(
                    Colors.white.withValues(alpha: 0.85),
                  ),
                ),
              ),
              const Spacer(),
              Padding(
                padding: const EdgeInsets.only(bottom: 28, left: 24, right: 24),
                child: Column(
                  children: [
                    const Text(
                      AppConstants.studentName,
                      style: TextStyle(
                        color: Colors.white,
                        fontWeight: FontWeight.w600,
                        fontSize: 14,
                      ),
                    ),
                    const SizedBox(height: 4),
                    Text(
                      '${AppConstants.studentNim} • ${AppConstants.studentClass}',
                      style: TextStyle(
                        color: Colors.white.withValues(alpha: 0.7),
                        fontSize: 12,
                      ),
                    ),
                  ],
                ),
              ),
```

```
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

lib/main.dart (11 baris)

```
import 'package:flutter/material.dart';
import 'package:intl/date_symbol_data_local.dart';

import 'app.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // Menyiapkan data locale Indonesia untuk format tanggal pada halaman detail.
  await initializeDateFormatting('id_ID');
  runApp(const RizalDocsApp());
}
```

lib/models/document.dart (84 baris)

```
/// Model data dokumen pada repositori Rizal Docs.
class Document {
  const Document({
    this.id,
    required this.title,
    required this.category,
    required this.author,
    required this.year,
    required this.description,
    required this.image,
    this.isFavorite = false,
    required this.createdAt,
    required this.updatedAt,
  });

  final int? id;
  final String title;
  final String category;
  final String author;
  final int year;
  final String description;

  /// Path aset lokal, mis. `assets/images/covers/cover_01.png`.
  final String image;
  final bool isFavorite;
  final DateTime createdAt;
  final DateTime updatedAt;

  factory Document.fromMap(Map<String, Object?> map) {
    return Document(
      id: map['id'] as int?,
      title: map['title'] as String,
      category: map['category'] as String,
      author: map['author'] as String,
      year: map['year'] as int,
      description: map['description'] as String,
      image: map['image'] as String,
      isFavorite: (map['is_favorite'] as int? ?? 0) == 1,
      createdAt: DateTime.parse(map['created_at'] as String),
      updatedAt: DateTime.parse(map['updated_at'] as String),
    );
  }

  Map<String, Object?> toMap() {
    return {
```

```
      'id': id,
      'title': title,
      'category': category,
      'author': author,
      'year': year,
      'description': description,
      'image': image,
      'is_favorite': isFavorite ? 1 : 0,
      'created_at': createdAt.toIso8601String(),
      'updated_at': updatedAt.toIso8601String(),
    };
  }

  Document copyWith({
    int? id,
    String? title,
    String? category,
    String? author,
    int? year,
    String? description,
    String? image,
    bool? isFavorite,
    DateTime? createdAt,
    DateTime? updatedAt,
  }) {
    return Document(
      id: id ?? this.id,
      title: title ?? this.title,
      category: category ?? this.category,
      author: author ?? this.author,
      year: year ?? this.year,
      description: description ?? this.description,
      image: image ?? this.image,
      isFavorite: isFavorite ?? this.isFavorite,
      createdAt: createdAt ?? this.createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
    );
  }
}
```

lib/models/user.dart (46 baris)

```
/// Model data pengguna aplikasi.
class User {
  const User({
    this.id,
    required this.nama,
    required this.username,
    required this.password,
    required this.createdAt,
  });

  final int? id;
  final String nama;
  final String username;
  final String password;
  final DateTime createdAt;

  factory User.fromMap(Map<String, Object?> map) {
    return User(
      id: map['id'] as int?,
      nama: map['nama'] as String,
      username: map['username'] as String,
      password: map['password'] as String,
      createdAt: DateTime.parse(map['created_at'] as String),
    );
  }

  Map<String, Object?> toMap() {
    return {
      'id': id,
      'nama': nama,
      'username': username,
      'password': password,
```

```
      'created_at': createdAt.toIso8601String(),
    };
  }

  User copyWith({int? id, String? nama, String? username, String? password}) {
    return User(
      id: id ?? this.id,
      nama: nama ?? this.nama,
      username: username ?? this.username,
      password: password ?? this.password,
      createdAt: createdAt,
    );
  }
}
```

lib/repositories/auth_repository.dart (60 baris)

```
import 'package:sqflite/sqflite.dart';

import '../core/database/database_helper.dart';
import '../models/user.dart';

/// Repository untuk operasi data pengguna pada database SQLite.
class AuthRepository {
  AuthRepository._();

  static final AuthRepository instance = AuthRepository._();

  /// Memeriksa kecocokan username dan password.
  ///
  /// Mengembalikan [User] jika cocok, atau `null` jika tidak ditemukan.
  Future<User?> login(String username, String password) async {
    final Database db = await DatabaseHelper.instance.database;
    final List<Map<String, Object?>> rows = await db.query(
      'users',
      where: 'username = ? AND password = ?',
      whereArgs: [username.trim().toLowerCase(), password],
      limit: 1,
    );

    if (rows.isEmpty) return null;
    return User.fromMap(rows.first);
  }

  /// Mengecek apakah username sudah dipakai.
  Future<bool> isUsernameTaken(String username) async {
    final Database db = await DatabaseHelper.instance.database;
    final List<Map<String, Object?>> rows = await db.query(
      'users',
      columns: ['id'],
      where: 'username = ?',
      whereArgs: [username.trim().toLowerCase()],
      limit: 1,
    );
    return rows.isNotEmpty;
  }

  /// Menyimpan pengguna baru dan mengembalikan data yang tersimpan.
  Future<User> register({
    required String nama,
    required String username,
    required String password,
  }) async {
    final Database db = await DatabaseHelper.instance.database;
    final User user = User(
      nama: nama.trim(),
      username: username.trim().toLowerCase(),
      // Catatan: password disimpan apa adanya untuk kebutuhan pembelajaran.
      // Pada aplikasi produksi gunakan hashing (mis. bcrypt/argon2).
      password: password,
      createdAt: DateTime.now(),
    );

    final int id = await db.insert('users', user.toMap()..remove('id'));
```

```
    return user.copyWith(id: id);
  }
}
```

lib/repositories/document_repository.dart (91 baris)

```
import 'package:sqflite/sqflite.dart';

import '../core/database/database_helper.dart';
import '../models/document.dart';

/// Repository untuk operasi CRUD dokumen pada database SQLite.
class DocumentRepository {
  DocumentRepository._();

  static final DocumentRepository instance = DocumentRepository._();

  Future<List<Document>> getAll() async {
    final Database db = await DatabaseHelper.instance.database;
    final List<Map<String, Object?>> rows = await db.query(
      'documents',
      orderBy: 'created_at DESC, id DESC',
    );
    return rows.map(Document.fromMap).toList();
  }

  Future<List<Document>> getFavorites() async {
    final Database db = await DatabaseHelper.instance.database;
    final List<Map<String, Object?>> rows = await db.query(
      'documents',
      where: 'is_favorite = ?',
      whereArgs: [1],
      orderBy: 'created_at DESC, id DESC',
    );
    return rows.map(Document.fromMap).toList();
  }

  Future<Document?> getById(int id) async {
    final Database db = await DatabaseHelper.instance.database;
    final List<Map<String, Object?>> rows = await db.query(
      'documents',
      where: 'id = ?',
      whereArgs: [id],
      limit: 1,
    );
    if (rows.isEmpty) return null;
    return Document.fromMap(rows.first);
  }

  Future<Document> create(Document document) async {
    final Database db = await DatabaseHelper.instance.database;
    final DateTime now = DateTime.now();
    final Document newDocument = document.copyWith(
      createdAt: now,
      updatedAt: now,
    );

    final int id = await db.insert(
      'documents',
      newDocument.toMap()..remove('id'),
    );
    return newDocument.copyWith(id: id);
  }

  Future<void> update(Document document) async {
    final int? id = document.id;
    if (id == null) {
      throw ArgumentError('Dokumen harus memiliki id untuk diperbarui.');
    }

    final Database db = await DatabaseHelper.instance.database;
    await db.update(
      'documents',
      document.copyWith(updatedAt: DateTime.now()).toMap(),
```

```
      where: 'id = ?',
      whereArgs: [id],
    );
  }

  Future<void> delete(int id) async {
    final Database db = await DatabaseHelper.instance.database;
    await db.delete('documents', where: 'id = ?', whereArgs: [id]);
  }

  Future<void> setFavorite(int id, bool isFavorite) async {
    final Database db = await DatabaseHelper.instance.database;
    await db.update(
      'documents',
      {
        'is_favorite': isFavorite ? 1 : 0,
        'updated_at': DateTime.now().toIso8601String(),
      },
      where: 'id = ?',
      whereArgs: [id],
    );
  }
}
```

pubspec.yaml (37 baris)

```
name: rizal_apps
description: "Rizal Docs - Aplikasi Repositori dan Koleksi Dokumen (UTS Mobile Programming)."
publish_to: 'none'

version: 1.0.0+1

environment:
  sdk: ^3.12.2

dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  # Penyimpanan lokal (SQLite) untuk data user dan dokumen.
  sqflite: ^2.4.2
  # Utilitas path untuk menentukan lokasi file database.
  path: ^1.9.1
  # Format tanggal Indonesia pada halaman detail dokumen.
  intl: ^0.20.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  integration_test:
    sdk: flutter

  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true

  # Seluruh gambar diambil dari aset lokal, bukan dari internet.
  assets:
    - assets/images/profile/
    - assets/images/brand/
    - assets/images/covers/
```

LAMPIRAN B · SOURCE CODE PENGUJIAN

Lampiran ini memuat unit test, widget test, dan integration test yang digunakan untuk memverifikasi aplikasi.

test/validators_test.dart (56 baris)

```
import 'package:flutter_test/flutter_test.dart';

import 'package:rizal_apps/core/utils/validators.dart';

void main() {
  group('AppValidators', () {
    test('nama wajib diisi dan minimal 3 karakter', () {
      expect(AppValidators.nama(null), 'Nama lengkap tidak boleh kosong');
      expect(AppValidators.nama('   '), 'Nama lengkap tidak boleh kosong');
      expect(AppValidators.nama('Ri'), 'Nama lengkap minimal 3 karakter');
      expect(AppValidators.nama('Rizal Suryawan'), isNull);
    });

    test('username wajib diisi', () {
      expect(AppValidators.username(''), 'Username tidak boleh kosong');
      expect(AppValidators.username('   '), 'Username tidak boleh kosong');
      expect(AppValidators.username('rizal'), isNull);
    });

    test('password wajib diisi dan minimal 6 karakter', () {
      expect(AppValidators.password(''), 'Password tidak boleh kosong');
      expect(AppValidators.password('12345'), 'Password minimal 6 karakter');
      expect(AppValidators.password('123456'), isNull);
    });

    test('konfirmasi password harus sama', () {
      expect(
        AppValidators.konfirmasiPassword('', 'rizal123'),
        'Konfirmasi password tidak boleh kosong',
      );
      expect(
        AppValidators.konfirmasiPassword('salah', 'rizal123'),
        'Konfirmasi password tidak sesuai',
      );
      expect(AppValidators.konfirmasiPassword('rizal123', 'rizal123'), isNull);
    });

    test('tahun harus berupa angka yang masuk akal', () {
      expect(AppValidators.tahun(''), 'Tahun tidak boleh kosong');
      expect(AppValidators.tahun('dua ribu'), 'Tahun harus berupa angka');
      expect(
        AppValidators.tahun('1800'),
        'Tahun harus di antara 1900 dan 2100',
      );
      expect(AppValidators.tahun('2025'), isNull);
    });

    test('requiredField memakai label pada pesan error', () {
      expect(
        AppValidators.requiredField('', 'Judul'),
        'Judul tidak boleh kosong',
      );
      expect(AppValidators.requiredField('Basis Data', 'Judul'), isNull);
    });
  });
}
```

test/document_model_test.dart (56 baris)

```
import 'package:flutter_test/flutter_test.dart';
```

```dart
import 'package:rizal_apps/models/document.dart';

void main() {
  group('Document', () {
    final DateTime createdAt = DateTime(2024, 6, 18, 10, 30);

    final Document document = Document(
      id: 1,
      title: 'Basis Data Relasional',
      category: 'Teknologi',
      author: 'Gusmayeni',
      year: 2022,
      description: 'Buku tentang basis data relasional.',
      image: 'assets/images/covers/cover_03.png',
      isFavorite: true,
      createdAt: createdAt,
      updatedAt: createdAt,
    );

    test('toMap dan fromMap menghasilkan data yang sama', () {
      final Map<String, Object?> map = document.toMap();
      final Document result = Document.fromMap(map);

      expect(map['is_favorite'], 1);
      expect(result.id, document.id);
      expect(result.title, document.title);
      expect(result.category, document.category);
      expect(result.author, document.author);
      expect(result.year, document.year);
      expect(result.description, document.description);
      expect(result.image, document.image);
      expect(result.isFavorite, isTrue);
      expect(result.createdAt, createdAt);
    });

    test('fromMap membaca is_favorite 0 sebagai false', () {
      final Map<String, Object?> map = document.toMap()..['is_favorite'] = 0;

      expect(Document.fromMap(map).isFavorite, isFalse);
    });

    test('copyWith hanya mengubah field yang diberikan', () {
      final Document updated = document.copyWith(
        title: 'Judul Baru',
        isFavorite: false,
      );

      expect(updated.title, 'Judul Baru');
      expect(updated.isFavorite, isFalse);
      expect(updated.author, document.author);
      expect(updated.createdAt, document.createdAt);
    });
  });
}
```

test/widget_test.dart (40 baris)

```dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

import 'package:rizal_apps/app.dart';
import 'package:rizal_apps/features/auth/login_page.dart';

void main() {
  testWidgets('splash screen berlanjut ke halaman login', (
    WidgetTester tester,
  ) async {
    await tester.pumpWidget(const RizalDocsApp());

    expect(find.text('Repositori & Koleksi Dokumen'), findsOneWidget);

    await tester.pump(const Duration(milliseconds: 1700));
    await tester.pumpAndSettle();
```

```
    expect(find.text('Masuk ke Akun'), findsOneWidget);
    expect(find.widgetWithText(FilledButton, 'Masuk'), findsOneWidget);
  });

  testWidgets('form login menampilkan pesan validasi saat kosong', (
    WidgetTester tester,
  ) async {
    await tester.pumpWidget(const MaterialApp(home: LoginPage()));

    final Finder submitButton = find.widgetWithText(FilledButton, 'Masuk');
    await tester.ensureVisible(submitButton);
    await tester.pumpAndSettle();

    // Memanggil aksi tombol secara langsung agar pengujian tidak bergantung
    // pada posisi scroll halaman.
    final FilledButton button = tester.widget<FilledButton>(submitButton);
    button.onPressed!();
    await tester.pump();

    expect(find.text('Username tidak boleh kosong'), findsOneWidget);
    expect(find.text('Password tidak boleh kosong'), findsOneWidget);
  });
}
```

integration_test/app_test.dart (168 baris)

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';

import 'package:rizal_apps/features/documents/widgets/document_card.dart';
import 'package:rizal_apps/main.dart' as app;

/// Pengujian end-to-end alur utama aplikasi:
/// registrasi -> login -> detail & favorit -> tab favorit -> tambah dokumen
/// -> ubah dokumen -> hapus dokumen.
void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  /// Menjalankan aksi tombol FilledButton secara langsung.
  ///
  /// Dipakai untuk tombol yang berada di dalam area scroll agar pengujian tidak
  /// bergantung pada posisi scroll maupun hit-testing pada perangkat.
  Future<void> pressFilledButton(WidgetTester tester, String label) async {
    final Finder finder = find.widgetWithText(FilledButton, label);
    final FilledButton button = tester.widget<FilledButton>(finder);
    expect(button.onPressed, isNotNull);
    button.onPressed!();
    await tester.pumpAndSettle();
  }

  testWidgets('alur registrasi, login, favorit, dan CRUD dokumen', (
    WidgetTester tester,
  ) async {
    final String unique = DateTime.now().millisecondsSinceEpoch.toString();
    final String username = 'rizal$unique';

    await app.main();
    await tester.pump(const Duration(seconds: 2));
    await tester.pumpAndSettle();

    // --- Registrasi ------------------------------------------------------
    await tester.tap(find.text('Registrasi Akun Baru'));
    await tester.pumpAndSettle();

    final Finder registerFields = find.byType(TextFormField);
    await tester.enterText(registerFields.at(0), 'Rizal Suryawan');
    await tester.enterText(registerFields.at(1), username);
    await tester.enterText(registerFields.at(2), 'rizal123');
    await tester.enterText(registerFields.at(3), 'rizal123');
    await pressFilledButton(tester, 'Daftar');

    // Kembali ke halaman login setelah registrasi berhasil.
    expect(find.text('Masuk ke Akun'), findsOneWidget);
```

```
// --- Login -------------------------------------------------------------
final Finder loginFields = find.byType(TextFormField);
await tester.enterText(loginFields.at(0), username);
await tester.enterText(loginFields.at(1), 'rizal123');
await tester.tap(find.widgetWithText(FilledButton, 'Masuk'));
await tester.pump();
await tester.pump(const Duration(milliseconds: 600));

expect(find.text('Login berhasil'), findsOneWidget);

await tester.pumpAndSettle();

// --- Dashboard ---------------------------------------------------------
expect(find.text('Koleksi Dokumen'), findsOneWidget);
expect(find.text('Halo, Rizal Suryawan'), findsOneWidget);

// --- Detail & favorit --------------------------------------------------
await tester.tap(find.byType(DocumentCard).first);
await tester.pumpAndSettle();

expect(find.text('Deskripsi'), findsOneWidget);

await tester.tap(find.text('Tambah ke Favorit'));
await tester.pumpAndSettle();

expect(find.byType(AlertDialog), findsOneWidget);
await tester.tap(
  find.descendant(of: find.byType(AlertDialog), matching: find.text('Ya')),
);
await tester.pump();
await tester.pump(const Duration(milliseconds: 600));

expect(
  find.text('Dokumen berhasil ditambahkan ke favorit'),
  findsOneWidget,
);
expect(find.text('Hapus dari Favorit'), findsOneWidget);

await tester.pageBack();
await tester.pumpAndSettle();

// --- Tab favorit -------------------------------------------------------
await tester.tap(
  find.descendant(
    of: find.byType(BottomNavigationBar),
    matching: find.byIcon(Icons.favorite_border_rounded),
  ),
);
await tester.pumpAndSettle();

expect(find.text('Favorit Saya'), findsOneWidget);
expect(find.byType(DocumentCard), findsWidgets);

// --- Menu pengaturan: create -------------------------------------------
await tester.tap(
  find.descendant(
    of: find.byType(BottomNavigationBar),
    matching: find.byIcon(Icons.settings_outlined),
  ),
);
await tester.pumpAndSettle();

expect(find.text('Daftar Dokumen'), findsOneWidget);

await tester.tap(find.widgetWithText(FilledButton, 'Tambah Dokumen'));
await tester.pumpAndSettle();

final Finder formFields = find.byType(TextFormField);
await tester.enterText(formFields.at(0), 'Dokumen Uji Otomatis');
await tester.enterText(formFields.at(1), 'Rizal Suryawan');
await tester.enterText(formFields.at(2), '2025');
await tester.enterText(
  formFields.at(3),
  'Dokumen ini dibuat otomatis oleh integration test.',
);
```

```
    await tester.tap(find.byType(FilledButton));
    await tester.pumpAndSettle();

    expect(find.text('Dokumen Uji Otomatis'), findsOneWidget);

    // --- Menu pengaturan: update ------------------------------------------
    await tester.tap(find.text('Dokumen Uji Otomatis'));
    await tester.pumpAndSettle();

    expect(find.text('Edit Dokumen'), findsOneWidget);

    final Finder editFields = find.byType(TextFormField);
    await tester.enterText(editFields.at(0), 'Dokumen Uji Diperbarui');
    await tester.tap(find.byType(FilledButton));
    await tester.pumpAndSettle();

    expect(find.text('Dokumen Uji Diperbarui'), findsOneWidget);
    expect(find.text('Dokumen Uji Otomatis'), findsNothing);

    // --- Menu pengaturan: delete ------------------------------------------
    final Finder updatedTile = find.ancestor(
      of: find.text('Dokumen Uji Diperbarui'),
      matching: find.byType(ListTile),
    );

    await tester.tap(
      find.descendant(
        of: updatedTile,
        matching: find.byType(PopupMenuButton<String>),
      ),
    );
    await tester.pumpAndSettle();
    await tester.tap(find.text('Hapus'));
    await tester.pumpAndSettle();

    await tester.tap(
      find.descendant(of: find.byType(AlertDialog), matching: find.text('Ya')),
    );
    await tester.pumpAndSettle();

    expect(find.text('Dokumen Uji Diperbarui'), findsNothing);
  });
}
```

LAMPIRAN C · REPOSITORI DAN RIWAYAT COMMIT

C.1 Repositori

Item	Keterangan
Alamat repositori	https://github.com/zalwan/flutter_application_1
Branch	`uts`
Author commit	zalwan <rizalsuryawan86@gmail.com>
Jumlah commit fitur	13 commit (di atas initial commit)

C.2 Riwayat Commit

```
b4f4d6a  docs: add README and application screenshots
8fba6eb  test: add unit, widget, and end-to-end tests
e7cf2e8  style(ui): keep status bar icons readable on dark headers
ed262b0  feat(favorites): show favorited documents from database
5a85e1b  feat(documents): add CRUD management through settings menu
c34d8f8  feat(documents): add detail page with favorite confirmation dialog
cd56497  feat(documents): seed documents from local assets and render list
860d9d8  fix(auth): center header content on authentication pages
cfbad4c  feat(dashboard): add header, scrollable body, and bottom navigation
7e44426  feat(auth): add SQLite login and registration with form validation
ebe1585  feat(assets): add locally generated profile, logo, and document covers
4b3bcf0  chore: rename project to rizal_apps and set up app skeleton
ba3fd18  chore: update analyzer excludes and lockfile
7069799  Initial commit: flutter_application_1
```

C.3 Catatan Keamanan

Password disimpan sebagai teks biasa pada database lokal karena fokus pembelajaran berada pada alur autentikasi dan validasi form. Pada aplikasi produksi, password wajib di-hash (misalnya bcrypt atau argon2) dan dikirim melalui koneksi aman.